

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Санкт-Петербургский политехнический университет Петра
Великого»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчет

**Научно-исследовательская работа (получение первичных
навыков научно-исследовательской работы)**

Выполнил:

студент группы 5130201/50301 _____ Таплыкова Е.В.

Принял:

канд. техн. наук., доцент ВШТИИ ИКНК _____ Глазунов В.В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	5
2 Особенности реализации	6
2.1 Установка и настройка среды виртуализации	6
2.2 Работа в командном интерпретаторе	9
2.3 Разработка статических и динамических библиотек	14
2.4 Исследование разделяемых библиотек	17
2.5 Двоичные файлы и визуализация в псевдографике	19
3 Результаты работы и эксперименты	32
Заключение	40
Список используемых источников	43
А. Исходный код файлов задания №1	44
В. Исходный код файлов задания №2	59

Введение

В рамках летней научно-исследовательской практики была выполнена работа по освоению операционной системы GNU/Linux Debian 12, инструментов разработки и систем программирования на языке Си. Основное внимание уделялось работе в среде виртуализации, изучению командного интерпретатора, а также разработке программного обеспечения с использованием статических и динамических библиотек, работе с двоичными файлами и псевдографической библиотекой ncurses.

В работе использовались следующие технологии и инструменты:

- VirtualBox – среда виртуализации для запуска гостевой операционной системы;
- GNU/Linux Debian 12 – операционная система, установленная на виртуальную машину;
- SSH – протокол для удаленного подключения к терминалу;
- CMake – мета-система сборки для управления процессом компиляции;
- ncurses – библиотека для создания псевдографических интерфейсов;
- zlib – библиотека для вычисления контрольной суммы CRC-32;
- GCC – компилятор языка Си;
- CMake – система сборки;
- Nano и Visual Studio Code – редакторы кода.

Практика направлена на закрепление навыков работы в Linux-среде, разработки на языке Си, управления зависимостями и сборкой проектов, а также

на исследование производительности программ при различных подходах к линковке.

1 Постановка задачи

Для того, чтобы считать работу выполненной необходимо закрыть следующие задачи:

1. Установить и настроить виртуальную среду Debian 12.14, выполнить проброс портов, настроить беспарольную SSH-аутентификацию;
2. Обучиться основам работы в командном интерпретаторе;
3. Разработать функцию проверки слова на палиндром в итеративном и рекурсивном вариантах, собрать статическую и динамическую библиотеки, реализовать линковку через `ldopen`;
4. Провести тестирование скорости работы программы при различных флагах оптимизации (`O0`, `O1`, `O2`, `Os`), сравнить размер результирующих файлов;
5. Провести исследование зависимостей с `ldd`;
6. Разработать структуры `PRICE` (категория, название, количество и цена товара) и заголовка базы данных (сигнатура, номер транзакции, количество структур в файле и контрольная сумма);
7. Разработать приложение с псевдографическим интерфейсом `ncurses`, реализовав функции добавления нового товара в базу данных, поиска средней цены товара с максимальным количеством в заданной категории, вывода содержимого базы данных и ее свойств;

2 Особенности реализации

2.1 Установка и настройка среды виртуализации

VirtualBox – это кроссплатформенное программное обеспечение для виртуализации, которое позволяет запускать несколько операционных систем (ОС) одновременно на одном физическом компьютере.

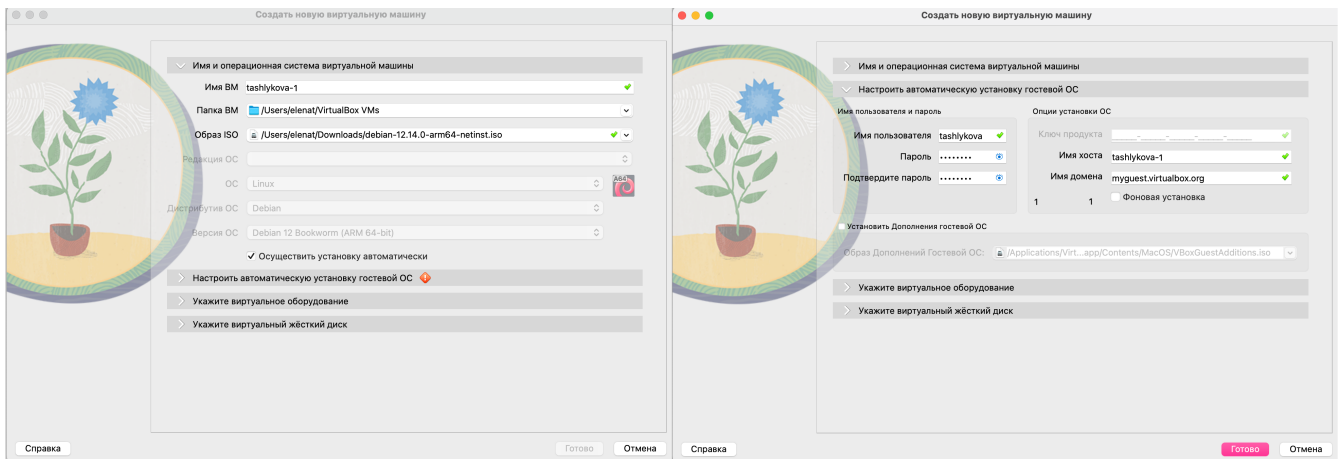


Рис. 1: Создание виртуальной машины

Главное условие установки ОС – создание логина повторяющего фамилию латиницей (*tashlykova*). Имя виртуальной машины и имя хоста – произвольные.

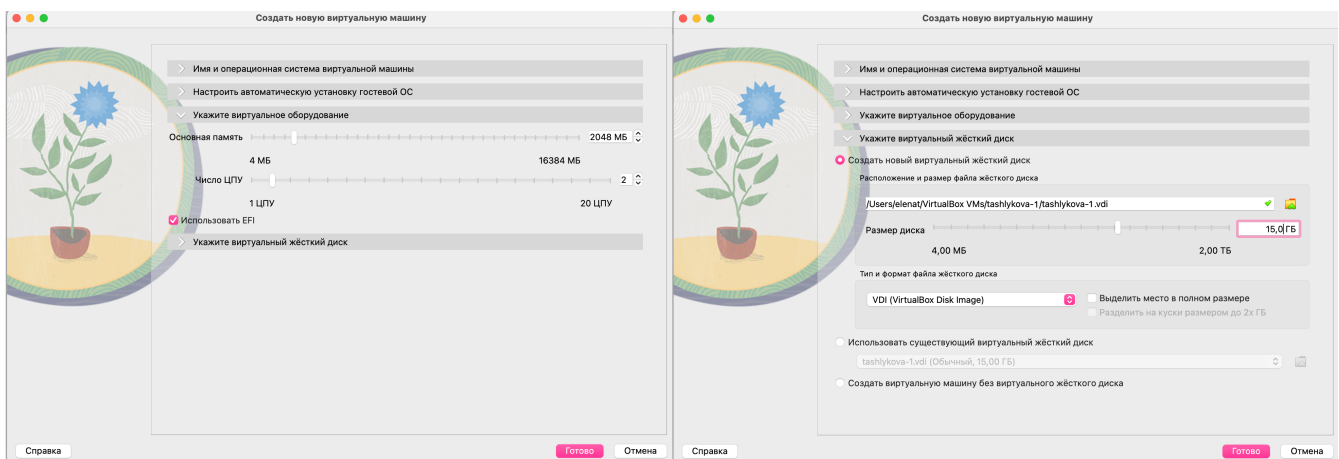


Рис. 2: Создание виртуальной машины

В качестве ISO-образа использовался *debian — 12.14.0 — arm64 — netinst*.

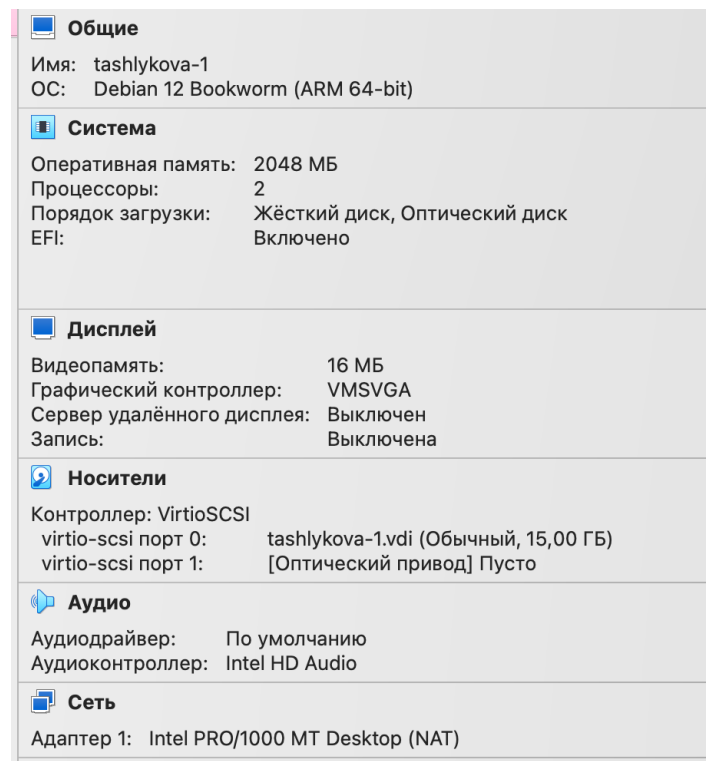


Рис. 3: Детали виртуальной машины

После загрузки системы на терминале `tty1` отображается приглашение к вводу имени пользователя и пароля.

```
Debian GNU/Linux 12 tashlykova-1 tty1
tashlykova-1 login: tashlykova
Password:
Linux tashlykova-1 6.1.0-49-arm64 #1 SMP Debian 6.1.174-1 (2026-05-26) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
last login: Wed Jun 10 17:07:40 MSK 2026 on tty1
tashlykova@tashlykova-1:~$ _
```

Рис. 4: Приглашение `tty1`

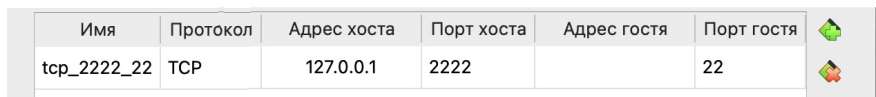
Однако аутентификация (процесс подтверждения личности пользователя) по паролю имеет множество недостатков. Если пароль недостаточно сложный,

то его можно подобрать перебором, а если же пароль сложный и уникальный, то его трудно запомнить. Существует еще недостатки, поэтому, чтобы избежать неприятностей мы будем использовать авторизацию с помощью SSH.

SSH (Secure Shell, «безопасная оболочка») – сетевой протокол для защищенного удаленного доступа к системе. Протокол обеспечивает шифрование соединения и проверку подлинности обеих сторон. SSH дает возможность подключиться к удаленному серверу и выполнять команды в терминале так же, как если бы использовалось локальное подключение на той же машине.

Для подключения к виртуальной машине с помощью встроенного SSH-клиента была выполнена настройка проброса портов (Port Forwarding):

- Тип подключения: NAT – режим сети, при котором виртуальная машина использует IP-адрес физического компьютера для выхода в интернет;
- Протокол: TCP – протокол управления передачей данных, который обеспечивает надёжное, установленное и упорядоченное соединение между ВМ и компьютером;
- Адрес хоста: 127.0.0.1;
- Порт хоста: 2222;
- Порт гостя: 22.



Имя	Протокол	Адрес хоста	Порт хоста	Адрес гостя	Порт гостя
tcp_2222_22	TCP	127.0.0.1	2222		22

Рис. 5: Проброс портов

Далее необходимо было сгенерировать пару ключей и публичный ключ

скопировать на виртуальную машину. Принцип работы SSH-ключей основан на асимметричном шифровании. Приватный ключ шифрует данные. Обратное преобразование может выполнить только связанный с ним публичный ключ.

```
ssh-keygen -t rsa -b 4096
ssh-copy-id -p 2222 tashlykova@127.0.0.1
ssh -p 2222 tashlykova@localhost
```

Благодаря этим действиям подключение к ВМ осуществляется без ввода пароля. Засчет такой аутентификации мы получаем стойкость к взлому, так как полный перебор всех возможных SSH-ключей невозможен даже с помощью сверхмощных современных компьютеров. Также приватный ключ никогда не покидает машину пользователя и не передается по сети, поэтому украсть его через сеть невозможно. Ключи не нужно запоминать, а одну пару можно использовать для доступа к своим разным серверам. Это дает нам удобство, гибкость и надежность при работе.

2.2 Работа в командном интерпретаторе

Интерпретатор предоставляет большое количество команд для работы. С помощью *man* я ознакомилась со справкой команд из списка задания, они приведены в таблице [1](#).

Чтобы получить справку, нужно ввести в терминале *man*, а через пробел – название команды, по которой нужна информация. Например, *man mkdir* выведет подробную документацию по команде *mkdir*.

```

MKDIR(1)                                     User Commands                               MKDIR(1)

NAME
  mkdir - make directories

SYNOPSIS
  mkdir [OPTION]... DIRECTORY...

DESCRIPTION
  Create the DIRECTORY(ies), if they do not already exist.

  Mandatory arguments to long options are mandatory for short options too.

  -m, --mode=MODE
        set file mode (as in chmod), not a=rwx - umask

  -p, --parents
        no error if existing, make parent directories as needed, with their file modes unaffected by any -m option.

  -v, --verbose
        print a message for each created directory

  -Z
        set SELinux security context of each created directory to the default type

  --context[=CTX]
        like -Z, or if CTX is specified then set the SELinux or SMACK security context to CTX

  --help
        display this help and exit

  --version
        output version information and exit

AUTHOR
  Written by David MacKenzie.

REPORTING BUGS
  GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
  Report any translation bugs to <https://translationproject.org/team/>

COPYRIGHT
  Copyright © 2022 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
  This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law.

SEE ALSO
  mkdir(2)

  Full documentation <https://www.gnu.org/software/coreutils/mkdir>
  or available locally via: info '(coreutils) mkdir invocation'

GNU coreutils 9.1                               September 2022                               MKDIR(1)
Manual page mkdir(1) line 1/50 (END) (press h for help or q to quit)

```

Рис. 6: Страница руководства команды mkdir

Каждая страница руководства разделена на логические блоки:

- NAME – название команды и краткое описание;
- SYNOPSIS – синтаксис команды, где перечислены все доступные опции;
- DESCRIPTION – подробное описание того, что делает команда;
- AUTHOR – кто написал или поддерживает данную команду;
- REPORTING BUGS – куда и как отправлять отчёты об ошибках, если вы нашли баг в работе команды;
- COPYRIGHT – информация о лицензии и юридических правах на программу;
- SEE ALSO – ссылки на связанные команды или дополнительные ресурсы;

```

[tashlykova@tashlykova-1:~/task1/build$ whoami
tashlykova
[tashlykova@tashlykova-1:~/task1/build$ cd ..
[tashlykova@tashlykova-1:~/task1$ cd ..
[tashlykova@tashlykova-1:~$ whoami
tashlykova
[tashlykova@tashlykova-1:~$ date
Sat Jun 20 03:41:15 PM MSK 2026
[tashlykova@tashlykova-1:~$ uptime
 15:41:34 up 19:16,  2 users,  load average: 0.00, 0.00, 0.00
[tashlykova@tashlykova-1:~$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
udev              960388          0      960388   0% /dev
tmpfs             202300          504      201796   1% /run
/dev/sda2        13873468 2187960  10958980  17% /
tmpfs            1011488          0     1011488   0% /dev/shm
tmpfs              5120           0         5120   0% /run/lock
/dev/sda1         524008         6208     517800   2% /boot/efi
tmpfs             202296          0      202296   0% /run/user/1000
[tashlykova@tashlykova-1:~$ touch test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 test.c tree.sh
[tashlykova@tashlykova-1:~$ rm test.c
[tashlykova@tashlykova-1:~$ ls
task1 task2 tree.sh
[tashlykova@tashlykova-1:~$ whereis main.c
[tashlykova@tashlykova-1:~$ cat task1/src/palindrome.h
// palindrome.h
#ifndef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

```

Рис. 7: Работа в командном интерпретаторе

С помощью *history* можно просмотреть всю историю команд. Зафиксируем три последние:

```

chmod +x scripts/*.sh
./scripts/analyze_libs.sh
./scripts/test_optimization.sh

```

Для дальнейшей работы необходимо установить группы пакетов разработки, для этого воспользуемся командой: *sudo apt install build — essential*. Пакет

build-essential включает gcc, make и всё для компиляции.

Так же установим мета-систему сборки cmake, с помощью команды: *sudo apt install CMake* (Cross-platform Make) — это кроссплатформенная система автоматизации сборки. Она генерирует файлы для сборки проекта на основе описания проекта в файлах *CMakeLists.txt*.

Далее было необходимо исследовать зависимости между программой командного интерпретатора */bin/bash* и динамическими библиотеками операционной системы при помощи команды *ldd*. Для этого построим рекурсивное дерево зависимостей. Для наглядности был разработан скрипт *tree.sh*.

```
# tree.sh
function print_deps() {
    local file=$1
    local indent=$2
    local first=$3
    if [ ! -f "$file" ]; then
        echo "${indent}[НЕ НАЙДЕНО: $file]"
        return
    fi
    ldd "$file" 2>/dev/null | while read line; do
        if echo "$line" | grep -q "linux-vdso"; then
            continue
        fi
        if echo "$line" | grep -q "=> /"; then
            lib=$(echo "$line" | awk '{print $3}')
            name=$(basename "$lib")
            echo "${indent}|-- $name"
            print_deps "$lib" "${indent}|" "no"
        elif echo "$line" | grep -q "~/lib/ld-linux"; then
            lib=$(echo "$line" | awk '{print $1}')
            name=$(basename "$lib")
            echo "${indent}|__ $name"
            print_deps "$lib" "${indent}" "no"
        fi
    fi
}
```

```

done
}
print_deps "$1" "" "yes"

```

Все программы зависят от libc.so.6 – стандартной библиотеки языка Си.

```

[tashlykova@tashlykova-1:~$ ./tree.sh /bin/bash
└─ libtinfo.so.6
   └─ libc.so.6
└─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /bin/ls
└─ libselinux.so.1
   └─ libpcre2-8.so.0
      └─ libc.so.6
   └─ libc.so.6
└─ libc.so.6
└─ libpcre2-8.so.0
   └─ libc.so.6
[tashlykova@tashlykova-1:~$ ./tree.sh /usr/bin/grep
└─ libpcre2-8.so.0
   └─ libc.so.6
└─ libc.so.6
tashlykova@tashlykova-1:~$ █

```

Рис. 8: Деревья зависимостей

В таблице 1 так же представлено количество зависимостей у исследуемых команд.

Таблица. 1: Основное назначение команды количество их зависимостей

Команда	Пояснение	Количество зави- симостей
man	Получение справочной информации коман- ды	12
whoami	Вывод имени текущего пользователя	1
date	Вывод текущей даты и времени	1
uptime	Время работы системы и нагрузка	11
df	Информация о свободном месте на дисках	1
cp	Копирование файлов	5
mv	Перемещение/переименование файлов	5
touch	Создание пустого файла	1
rm	Удаление файлов	1
stat	Информация о файле (размер, права, вре- мя)	3
file	Определение типа файла	5
find	Поиск файлов по критериям	4
cat	Вывод содержимого файла	1
hexdump	Просмотр файла в шестнадцатеричном ви- де	2
less	Постраничный просмотр файла	2
time	Измерение времени выполнения команды	3

2.3 Разработка статических и динамических библиотек

В индивидуальном задании №1 было необходимо создать библиотеку с реализацией функции, которая проверяет является ли указанное слово палиндромом, при этом она должна быть реализована различными способами: с рекурсией и без рекурсии, также в реализации запрещалось использовать библиотечные функции. Для этого я написала две основные функции *palindrome*, *palindrome_recursive* – итеративная и рекурсивная реализация, а так же одну вспомогательную функцию *string_len*, для вычисления длины строки.

```

#ifdef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

bool palindrome(char *string, int n);
bool palindrome_recursive(char *string, int n);
int string_len(char *string);

#endif

```

Алгоритм *palindrome*: сравниваются символы с начала и конца строки, постепенно сдвигаясь к центру. При обнаружении несовпадающей пары возвращается false. Время работы $O(n/2)$, память $O(1)$.

Алгоритм *palindrome_recursive*: базовый случай – пустая строка или строка из одного символа являются палиндромами. Иначе сравниваются первый и последний символы строки; при совпадении выполняется рекурсивный вызов для подстроки без крайних символов. Время работы $O(n)$, память $O(n)$. Исходный код приведен в Приложении А.

Для управления сборкой библиотек и тестовых программ использовалась мета-система сборки CMake. Структура проекта выглядит следующим образом:

```

task1
----CMakeLists.txt
----src/
    |---CMakeLists.txt
    |---palindrome.c
    |---palindrome.h
----tests/
    |---CMakeLists.txt

```

```
|---test_palindrome.c
|---test_dlopen.c
----scripts/
|---analyze_libs.sh
|----test_optimization.sh
```

Статическая библиотека – это коллекция объектных файлов (.o), объединенных в один архивный файл. Во время компиляции программы, использующей статическую библиотеку, компоновщик (linker) копирует весь необходимый код из библиотеки в исполняемый файл. Таким образом, исполняемый файл становится самодостаточным и не зависит от наличия библиотеки в системе во время выполнения.

Динамическая (разделяемая) библиотека – это коллекция скомпилированного кода, который загружается в память программы во время запуска. Исполняемый файл содержит только ссылки на функции и данные этой библиотеки, а не их копии. Это позволяет нескольким программам использовать одну и ту же копию библиотеки в памяти.

Настройка сборки библиотек в *src/CMakeLists.txt*:

```
# Статическая библиотека
add_library(palindrome_static STATIC ${LIB_SOURCES})
set_target_properties(palindrome_static PROPERTIES
    OUTPUT_NAME palindrome
)

# Динамическая библиотека
add_library(palindrome_shared SHARED ${LIB_SOURCES})
set_target_properties(palindrome_shared PROPERTIES
    OUTPUT_NAME palindrome
    VERSION ${PROJECT_VERSION}
    SOVERSION 1
```


)

При обычной компоновке с динамической библиотекой, ОС автоматически загружает все необходимые библиотеки при запуске программы. Если какая-либо из них отсутствует, программа не запустится.

Однако существует еще более гибкий механизм работы с динамическими библиотеками – их динамическая загрузка во время выполнения (runtime loading). Runtime loading предоставляет программисту полный контроль над процессом, это означает, что программа может решить, какую библиотеку загрузить и какие функции из нее использовать, не на этапе компиляции, а уже в процессе своей работы. Для реализации этого механизма в Unix/Linux используется набор функций, предоставляемых библиотекой dl (dynamic linking library)

Для загрузки библиотеки во время выполнения использованы функции `dlopen()` и `dlsym()`:

```
void *handle = dlopen("libpalindrome.so", RTLD_LAZY);  
palindrome_func_t func = (palindrome_func_t)dlsym(handle, "palindrome");
```

2.4 Исследование разделяемых библиотек

Как происходит тестирование функций *palindrome* и *palindrome_recursive*:

1. Генерируются случайные строки заданной длины;
2. Для каждой строки вычисляется длина с помощью *string_len*;
3. Выполняется проверка на палиндром (итеративная или рекурсивная);
4. Измеряется процессорное время с помощью *clock()*;
5. Результат усредняется по количеству итераций.

Тестирование проходит для каждого типа линковки библиотеки. Также дополнительно разработан скрипт для автоматического тестирования всех флагов оптимизации:

- -O0 – без оптимизации (режим отладки)
- -O1 – базовые оптимизации
- -O2 – полная оптимизация (рекомендуемый уровень)
- -Os – оптимизация по размеру кода

Исследуем разделяемые библиотеки в полученной программе, используя команду `ldd` и построим соответствующие рекурсивные деревья зависимостей.

```
[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_palindrome
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_palindrome_dynamic
├── libpalindrome.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/bin/test_dlopen
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
[tashlykova@tashlykova-1:~$ ./tree.sh task1/build_analysis/lib/libpalindrome.so
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1
```

Рис. 9: Дерево зависимостей

Статическая линковка (*test_palindrome*)

Программа содержит только 2 зависимости: `libc.so.6` (стандартная библиотека Си) и `ld-linux-aarch64.so.1` (динамический загрузчик). Библиотека `libpalindrome` не отображается в зависимостях, так как ее код был скопирован в исполняемый

файл на этапе компиляции.

Динамическая линковка (*test_palindrome_dynamic*)

Программа содержит 3 зависимости: `libc.so.6`, `ld-linux-aarch64.so.1`, `libpalindrome.so.1` (динамическая библиотека с функциями палиндрома). Библиотека `libpalindrome` загружается при запуске программы. Если библиотека не будет найдена в системе, программа не запустится.

Динамическая загрузка через `dlopen` (*test_dlopen*)

Программа содержит только 2 зависимости: `libc.so.6` и `ld-linux-aarch64.so.1`. Отметим, что `libpalindrome.so` не отображается в выводе `ldd`, так как загружается динамически во время выполнения через функции `dlopen()` и `dlsym()`. Это позволяет программе загружать библиотеку по требованию и обрабатывать ситуацию ее отсутствия.

2.5 Двоичные файлы и визуализация в псевдографике

Следующий этап научно-исследовательской работы – разработка программы с использованием псевдографической библиотеки `ncurses`.

Ncurses («**new curses**») – это библиотека предназначенная для управления вводом-выводом на терминал. Она позволяет работать с виртуальным экраном, создавать и управлять несколькими окнами на одном терминале, каждое со своим содержимым и атрибутами.

Весь интерфейс построен на основе оконной модели – каждый экран или диалог представляет собой отдельное окно, что позволяет гибко управлять отображением и обновлением информации. Для реализации были использованы следующие функции `ncurses`:

- *initscr()* – инициализирует библиотеку ncurses и создает стандартное окно (stdscr), представляющее весь терминал. Вызывается в начале каждой функции, работающей с интерфейсом.
- *newwin(height, width, start_y, start_x)* – создает новое окно заданного размера в указанной позиции. Используется для создания модальных диалогов и меню.
- *delwin(win)* – удаляет созданное окно и освобождает память.
- *box(win, 0, 0)* – рисует рамку вокруг окна с использованием символов по умолчанию (ASCII-графика).
- *mvwprintw(win, y, x, format, ...)* – используется для позиционированного вывода форматированной строки.
- *endwin()* – завершает работу с ncurses и восстанавливает терминал в исходное состояние. Вызывается перед выходом из любой функции интерфейса.
- *clear()* – очищает стандартное окно, удаляя весь предыдущий вывод.
- *refresh()* – обновляет физический экран, синхронизируя буфер ncurses с реальным терминалом.

Приведу пример создания главного дисплея.

```
WINDOW *main_win = newwin(17, 55, (rows - 17) / 2, (cols - 55) / 2);
box(main_win, 0, 0);
mvwprintw(main_win, 0, 2, " PRODUCT DATABASE MANAGEMENT ");
```

Для ввода данных используются функции с поддержкой редактирования:

- *wgetch(win)* – получает один символ из окна (неблокирующий ввод с обработкой специальных клавиш).

- *wgetnstr(win, str, size)* – считывает строку текста с клавиатуры.
- *echo()* – включает отображение вводимых символов на экране.
- *noecho()* – отключает отображение вводимых символов.
- *cbreak()* – переводит терминал в режим посимвольного ввода, заставляя терминал передавать каждый введённый символ программе сразу (без буферизации).
- *curs_set(0/1)* – управляет видимостью курсора (0 – скрыт, 1 – видим).
- *wmove(win, y, x)* – перемещает курсор в указанную позицию в окне.
- *wclrtoeol(win)* – очищает строку от текущей позиции курсора до конца.

Программа на ncurses

В рамках задания было необходимо реализовать структуру с именем PRICE, содержащую следующие поля:

- Название категории (строка Си – указатель на символ char – 8 байт).
- Название товара (строка Си – указатель на символ char – 8 байт).
- Стоимость товара в рублях (беззнаковый вещественный тип с точностью до двух знаков после запятой – 8 байт).
- Количество товаров в наличии (беззнаковый целочисленный тип – 4 байта).

А так же создать структуру описывающую заголовок базы данных, содержащую:

- Сигнатура формата файла (4 байта) – массив символов char, содержит первые 4 буквы фамилии латиницей.

- Номер транзакции (4 байта) – беззнаковое целое число, которое должно увеличиваться при каждом обращении к файлу, как при чтении, так и при записи.
- Количество структур в файле (4 байта) – беззнаковое целое число структур данных PRICE следующих за заголовком.
- Контрольная сумма (4 байта) – беззнаковое целое число, контрольная сумма всех данных следующих за заголовком по алгоритму CRC-32 вызываемая из библиотеки zlib.h

Далее представлена структура *PRICE*, а так же структура для заголовка базы данных *DB_HEADER*.

```
typedef struct
{
    char *category;
    char *good;
    double price;
    unsigned int num;
} PRICE;

typedef struct
{
    char signature[4];
    unsigned int tr_num;
    unsigned int st_count;
    unsigned int crc32;
} DB_HEADER;
```

В начале программы нас приветствует меню «product database management» состоящее из 6 полей, при вводе числа от 1 до 6 можно выполнить одну из команд.

```
PRODUCT DATABASE MANAGEMENT
1. Add new product
2. Search by category
3. Show all products
4. Save database
5. Database info
6. Exit

Records in memory: 0
Transaction #: 0

No database found. Creating new. CRC-32: 0x00000000

Select action (1-6): █
```

Рис. 10: Стартовое меню

Чтобы добавить новый продукт необходимо ввести цифру **1**. Для этого реализована функция `int input_data_ncurses()`. Вначале она рисует окно размером 14x55 символов с рамкой, в котором последовательно запрашивает четыре поля: категорию, название товара, цену и количество. Далее идет обработка вводимых данных. Чтобы получить символ с клавиатуры используется цикл с `ch = wgetch(win);`, это позволяет обрабатывать символы по одному, учитывая специальные клавиши: delete (backspace), esc и return (enter). После завершения ввода всех полей выполняется валидация: проверяется, что строки не пустые, цена и количество являются положительными числами. При обнаружении ошибки в нижней части окна выводится сообщение с пояснением, и ввод запрашивается заново. Функция возвращает целое число: 1 – если запись произведена успешна, 0 – если новый продукт не был добавлен (например, пользователь нажал esc).

```
ENTER PRODUCT DATA
Category:  milk
Product name: 
Press ESC to cancel

ENTER PRODUCT DATA
Category:  milk
Product name: cheese
Cost (rub): 12.56
Amount: 10
Press ESC to cancel
```

Рис. 11: Добавление нового продукта

Если все данные введены корректно, то пользователь получает сообщение, что новый продукт успешно записан в базу.

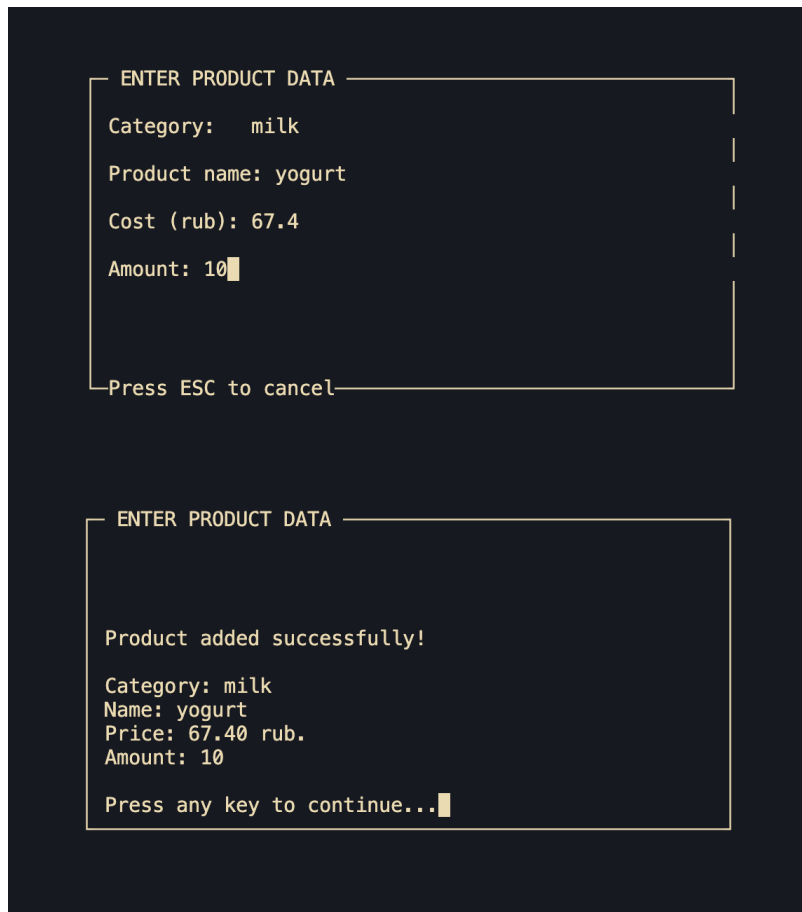


Рис. 12: Добавление нового продукта

После записи пользователь возвращается в исходное меню, теперь введем цифру **3**, чтобы убедиться, что все продукты были добавлены. С помощью функции *void show_all_products()* программа отображает в псевдографическом режиме список всех товаров, хранящихся в оперативной памяти. Она создает окно с рамкой и последовательно выводит информацию о каждом товаре: номер записи, категорию, название, цену и количество. Если база данных пуста, выводится соответствующее сообщение – «Database is empty!». Для вывода используется функция *mvprintw()*, позволяющая позиционировать текст в заданных координатах экрана. Просмотр завершается по нажатию любой клавиши.

```
ALL PRODUCTS
[1] Category: milk
    Name: cheese
    Price: 12.56 rub.
    Amount: 10
[2] Category: milk
    Name: yogurt
    Price: 67.40 rub.
    Amount: 10

Total: 2 products. Press any key...
```

Рис. 13: Просмотр всех продуктов в базе

При вводе цифры **4** можно сохранить базу данных, после этого значение контрольной суммы CRC-32 изменится. Сохранение происходит за счет функции *void save_with_message(char *signature)*. Перед сохранением программа проверяет наличие записей в памяти, при пустой базе выводится предупреждение. В процессе сохранения обновляется заголовок базы данных (количество записей и CRC-32): полю *st_count* (количество записей) присваивается текущее значение *item_count*, вызывается функция *create_header()* для пересчета контрольной суммы CRC-32 и формирования актуального заголовка. Далее вызывается функция *save_database()*, которая записывает заголовок и все структуры в файл в бинарном формате.

Файл состоит из двух логических частей:

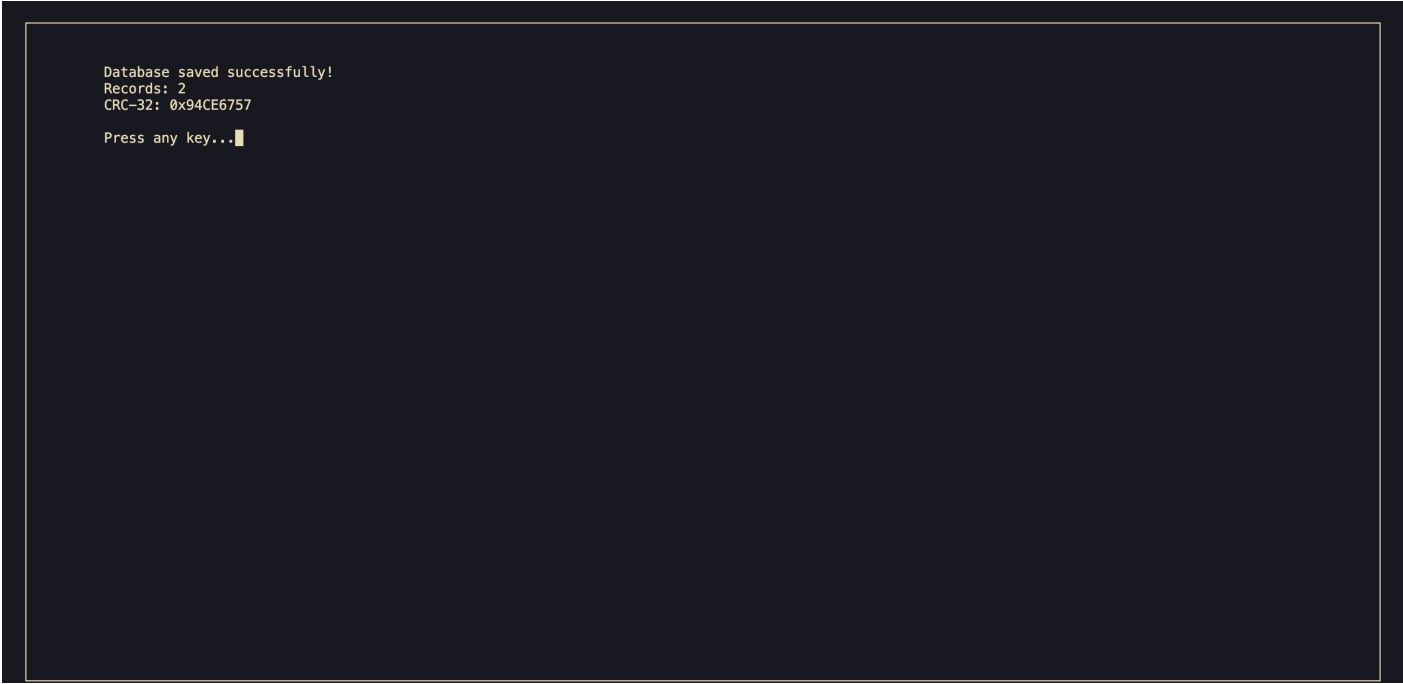
1. **Заголовок** (16 байт) – содержит сигнатуру формата, номер транзакции, количество записей и контрольную сумму;

2. **Данные** (переменный размер) – последовательность записей *PRICE*.

Каждая запись *PRICE* сохраняется в следующем формате:

- Длина строки категории (включая завершающий нулевой символ);
- Строка категории;
- Длина строки названия товара (включая завершающий нулевой символ);
- Строка названия товара;
- Цена товара;
- Количество товара.

При успешном сохранении пользователю отображается подтверждение с количеством записей и значением контрольной суммы.



```
Database saved successfully!  
Records: 2  
CRC-32: 0x94CE6757  
Press any key...■
```

Рис. 14: Сохранение базы данных

Чтобы просмотреть больше информации о базе данных, необходимо вве-

сти цифру 5. Функция *void show_database_info()* позволит узнать сигнатуру формата файла, номер транзакции, количество структур в файле, количество записей, которые загружены в оперативную память, контрольную сумму, а также файл, в котором хранятся данные.

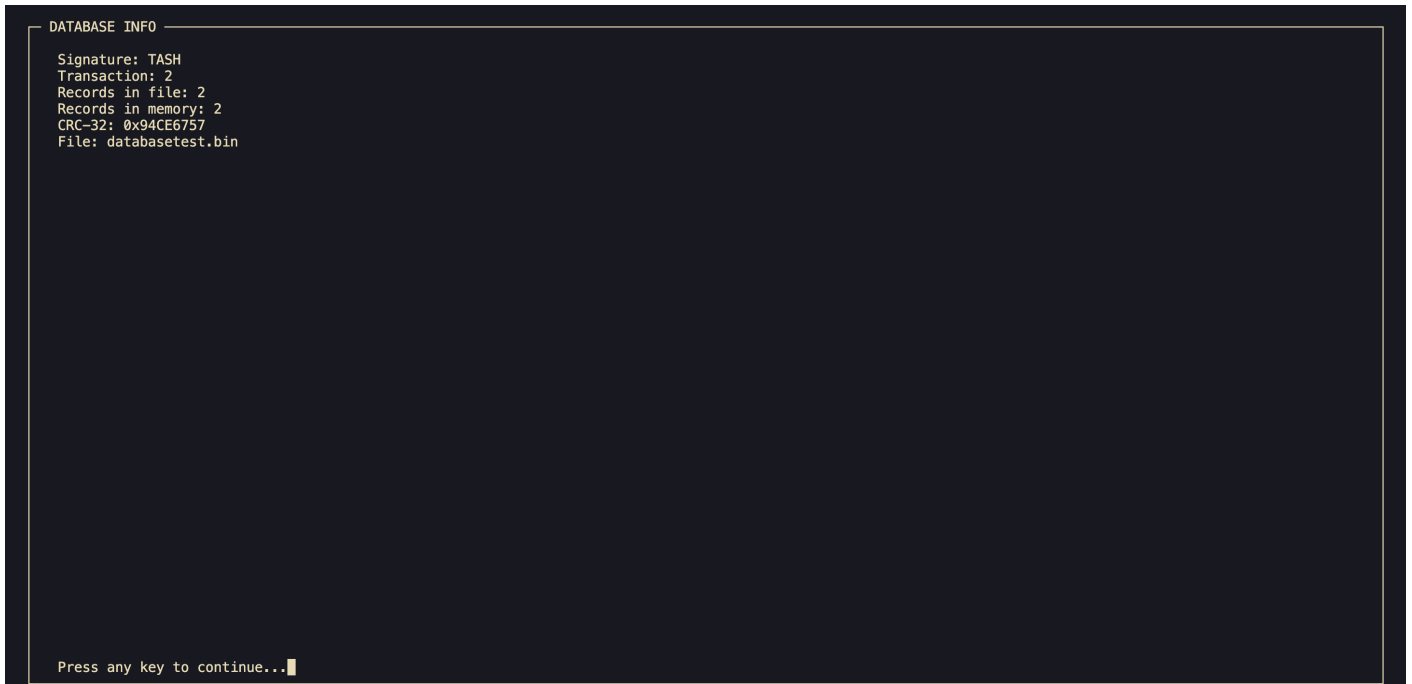
A screenshot of a terminal window with a dark background. The title bar of the window reads "DATABASE INFO". Inside the terminal, the following text is displayed: "Signature: TASH", "Transaction: 2", "Records in file: 2", "Records in memory: 2", "CRC-32: 0x94CE6757", and "File: databasetest.bin". At the bottom of the terminal, there is a prompt that says "Press any key to continue..." followed by a cursor.

Рис. 15: Просмотр дополнительной информации о базе данных

Перейдем к следующему индивидуальному заданию, в котором необходимо найти и вывести информацию о средней цене товаров с максимальным количеством, в категории, значение которой введено с клавиатуры. Для этого введем **2** в основном меню программы. Вызывается функция *int search_category_ncurses()*, которая отрисовывает окно для ввода категории, последовательно считывает символы, включая учитывая специальные клавиши: delete (backspace), esc и return (enter). Далее функция *void search_by_category(const char *category)* получает категорию и выполняет поиск, для каждой записи проверяется совпадение категории, находится максимальное количество товара в категории и запоминаются все товары с этим количеством и вычисляется средняя цена то-

варов с максимальным количеством. Результаты выводятся на экран. Функция `int search_category_ncurses()` возвращает целое значение 0 – если пользователь отменил или не ввел категорию, или 1 – если поиск осуществился.

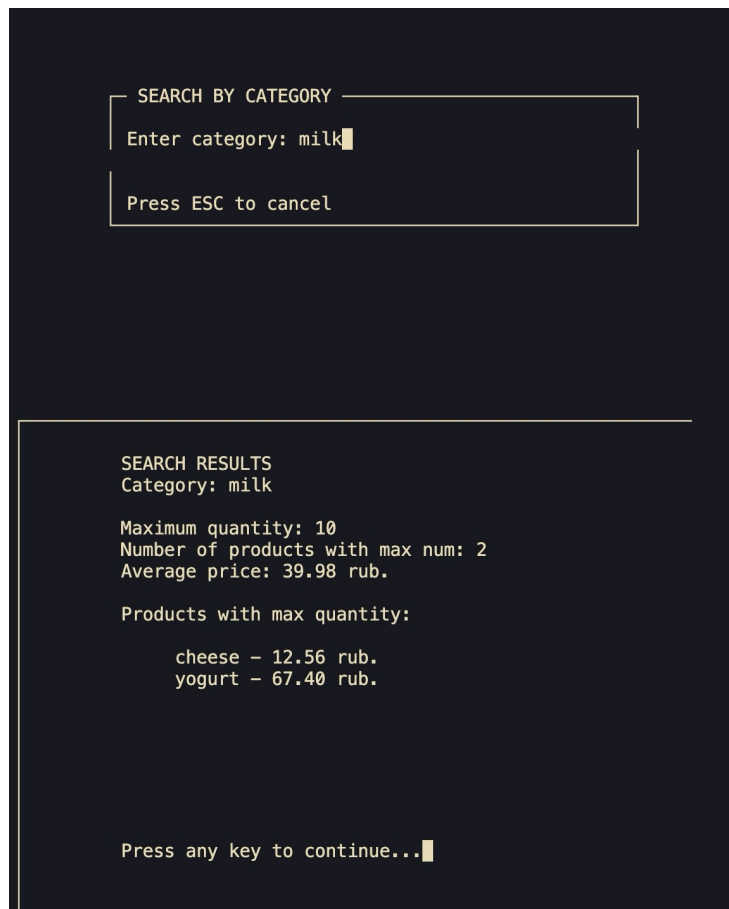


Рис. 16: Поиск по существующей категории

Если такого товара нет, на дисплее выдается соответствующее сообщение.

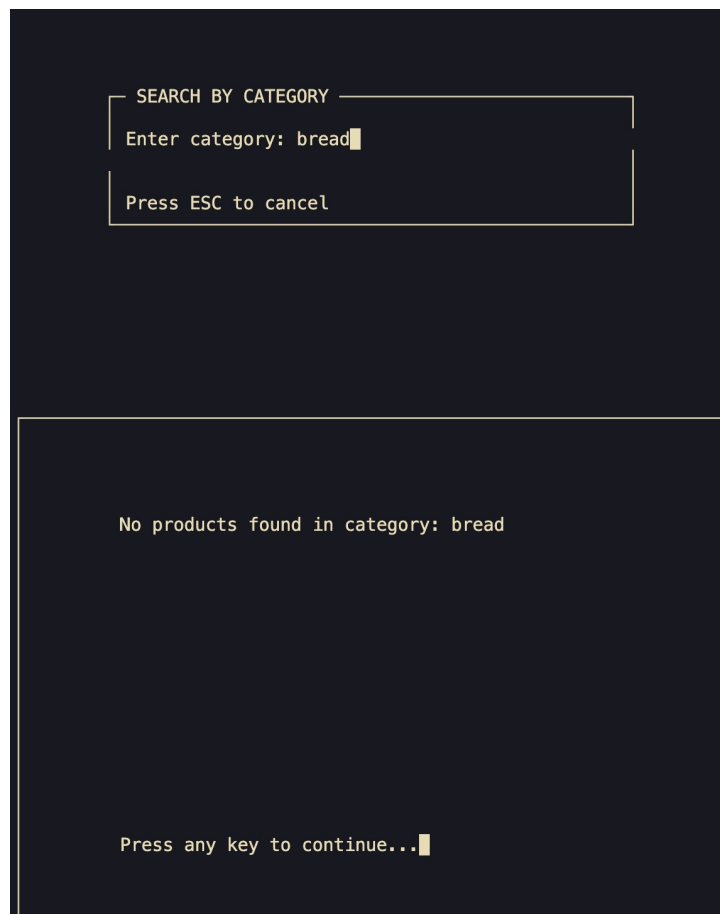


Рис. 17: Поиск по не существующей категории

После выполнения задания по разработке программы необходимо исследовать разделяемые библиотеки в полученной программе, используя команду `ldd` и построить соответствующее рекурсивное дерево зависимостей.

```

[tashlykova@tashlykova-1:~/task2/build$ ../../tree.sh price_db
├── libncurses.so.6
│   ├── libtinfo.so.6
│   │   ├── libc.so.6
│   │   │   └── ld-linux-aarch64.so.1
│   │   └── ld-linux-aarch64.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libtinfo.so.6
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libz.so.1
│   ├── libc.so.6
│   │   └── ld-linux-aarch64.so.1
│   └── ld-linux-aarch64.so.1
├── libc.so.6
│   └── ld-linux-aarch64.so.1
└── ld-linux-aarch64.so.1

```

Рис. 18: Дерево зависимостей

Были выявлены следующие динамические зависимости:

- `libncurses.so.6` – основная библиотека для создания псевдографического интерфейса. Предоставляет все необходимые функции для работы с окнами, вводом и выводом в терминале.
- `libtinfo.so.6` – библиотека терминальной информации (`terminfo`). Обеспечивает корректную работу с различными типами терминалов. Загружается как зависимость `libncurses`.
- `libz.so.1` – библиотека сжатия данных `zlib`. Используется в программе для вычисления контрольной суммы CRC-32 заголовка базы данных.
- `libc.so.6` – стандартная библиотека языка Си (`GNU C Library`). Предоставляет базовые функции для работы с памятью, файлами, строками и математическими операциями.

- `ld-linux-aarch64.so.1` – динамический загрузчик. Отвечает за загрузку и связывание всех динамических библиотек при запуске программы. Является обязательной зависимостью для всех динамически слинкованных программ в Linux.

3 Результаты работы и эксперименты

Проведем тестирование времени и корректности выполнения функций *palindrome* и *palindrome_recursive* на собственных и случайно сгенерированных данных.

```
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome 1000000 10000 iter
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome 1000000 10000 recursive

tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome_dynamic 1000000 10000 iter
tashlykova@tashlykova-1:~/task1/build$ ./bin/test_palindrome_dynamic 1000000 10000 recursive

tashlykova@tashlykova-1:~/task1/build$ ./bin/test_dlopen 1000000 10000
```

Для генерируемых данных количество итераций: 1000000, длина слова: 10000. Используются 3 типа линковки: статическая, динамическая, динамическая загрузка (`dlopen`).

Таблица. 2: Тестирование корректности функций

Слово	Длина	Ожидаемый результат	Фактический результат
level...level	135	true	true
hello...hello	55	false	false
bebe...eb	113	true	true
aAA...AAa	112	true	true
friends...friends	126	false	false
A	1	true	true
aA	2	false	false
(пустая строка)	0	true	true

Таким образом, ожидаемый результат совпадает с фактическим результатом, значит функции работают корректно.

Таблица. 3: Тестирование скорости работы функций

Тип линковки	Метод	Время выполнения (с)	Среднее время на слово (мс)
Статическая линковка	итеративный	45.857222	0.045857
Статическая линковка	рекурсивный	46.009406	0.046009
Динамическая линковка	итеративный	46.192764	0.046193
Динамическая линковка	рекурсивный	45.730578	0.045731
Динамическая загрузка через dlopen	итеративный	46.294545	0.046295
Динамическая загрузка через dlopen	рекурсивный	46.695704	0.046696

Таким образом при статической линковке рекурсивный метод медленнее на 0.33%, при динамической – рекурсивный метод быстрее на 1.0%, при динамической загрузке с помощью dlopen – рекурсивный метод медленнее на 0.86%. Можно считать разницу незначительной (дополнительные флаги для оптимизации не используются).

Следующим проведем тестирование с разными флагами компиляции (-O0, -O1, -O2, -Os).

```
./scripts/analyze_libs.sh
```

```
./scripts/test_optimization.sh
```

Таблица. 4: Время выполнения программы с различными флагами оптимизации

Флаг	Итеративный (стат.), с.	Рекурсивный (стат.), с.	Итеративный (дин.), с.	Рекурсивный (дин.), с.	dlopen (итер.), с.	dlopen (рек.), с.
O0	4.574	4.612	4.610	4.627	4.638	4.650
O1	4.106	4.119	4.130	4.174	4.126	4.140
O2	3.923	3.896	3.900	3.923	3.896	3.895
Os	4.641	4.626	4.640	4.643	4.627	4.631

Проведем анализ полученных результатов. Флаг -O0 дает самое медленное выполнение программы, при таком подходе код компилируется без каких-либо оптимизаций, все переменные хранятся в памяти.

Флаг -O1 обеспечивает хороший баланс между временем компиляции и производительностью, время выполнения программы немного уменьшается. С таким подходом идет устранение мертвого кода, присутствует оптимизация переходов, имеется начальное распределение переменных по регистрам.

Флаг -O2 дал наилучшую производительность по времени, благодаря развертыванию циклов, оптимизации хвостовой рекурсии, векторизации операций.

Флаг -Os уменьшает размер исполняемого кода, использует более короткие инструкции. Время выполнения программы, так же как и с -O0, медленное.

Далее проведем анализ размеров библиотек, учитывая флаги компиляции.

Таблица. 5: Размеры библиотек

Флаг	Размер <i>libpalindrome.a</i> , К
-O0	2.1
-O1	1.9
-O2	2.0
-Os	1.8

Наименьший размер занимает библиотека при компиляции с флагом -Os, больше всего – с -O0. При этом все исполняемые файлы имеют одинаковый размер (70К) независимо от флага оптимизации (размеры файлов автоматически сохраняются в папку results/).

Опираясь на таблицы 4 и 5, можно сделать следующие выводы:

1. O2 – лучший выбор для максимальной производительности;
2. Os – предпочтителен для систем с ограниченным дисковым пространством;
3. O1 – хороший компромисс для разработки;
4. O0 – используется только для отладки.

Далее проверим работоспособность программы с псевдографическим интерфейсом, для этого будем вводить некорректные данные.

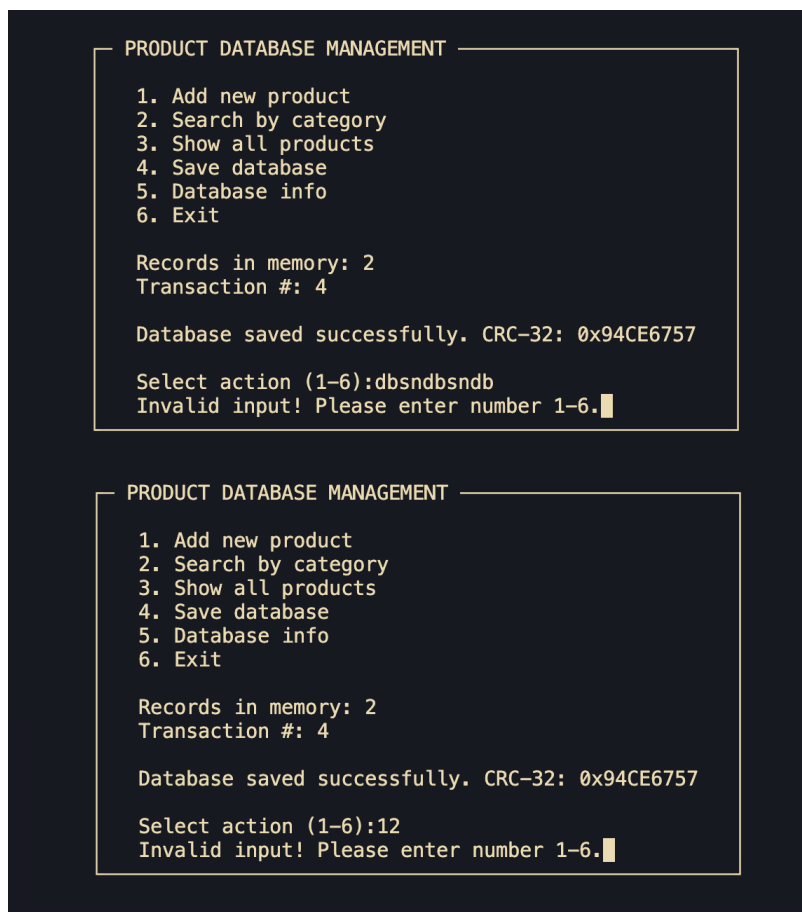


Рис. 19: Ввод некорректных данных при создании нового объекта

Как можно увидеть, в программе предусмотрена защита от ввода некорректных значений, таких как : ввод пустого поля вместо категории или названия продукта, ввод не числовых значений, значений меньше 0 в поля стоимости и количества товара. Для удобства внизу окна выводится причина ошибки. Стоит

отметить, что в поле с ценой пользователь может вводить только цифры, буквы и точку, в поле с количеством – только цифры и буквы. Другие специальные символы автоматически убираются программой.

Также в программе реализована защита от ввода некорректных данных в основном меню.



```
PRODUCT DATABASE MANAGEMENT
1. Add new product
2. Search by category
3. Show all products
4. Save database
5. Database info
6. Exit

Records in memory: 2
Transaction #: 4

Database saved successfully. CRC-32: 0x94CE6757

Select action (1-6):dbsndbsndb
Invalid input! Please enter number 1-6.

PRODUCT DATABASE MANAGEMENT
1. Add new product
2. Search by category
3. Show all products
4. Save database
5. Database info
6. Exit

Records in memory: 2
Transaction #: 4

Database saved successfully. CRC-32: 0x94CE6757

Select action (1-6):12
Invalid input! Please enter number 1-6.
```

Рис. 20: Ввод некорректных данных при выборе команды из основного меню

При вводе неподходящих значений после нажатия ENTER программа пишет сообщение об ошибке, ждет 1 секунду (*narpm*(1000);) и очищает поле для ввода.

Чтобы посмотреть корректность инициализации и изменения сигнатуры заголовка базы данных воспользуемся командой *hexdump*.

```
elenat@MacBook-Pro-elenatashlykova build % hexdump databasetest.bin
00000000 4154 4853 0010 0000 0004 0000 9c39 a9f6
00000010 0005 0000 696d 6b6c 0700 0000 6300 6568
00000020 7365 0065 851f 51eb 1eb8 4029 000a 0000
00000030 0005 0000 696d 6b6c 0700 0000 7900 676f
00000040 7275 0074 999a 9999 d999 4050 000a 0000
00000050 0007 0000 7773 6565 7374 0800 0000 6300
00000060 6f6f 696b 7365 0000 0000 0000 2900 7840
00000070 0000 0800 0000 6300 6f6c 6874 7365 0600
00000080 0000 6a00 6165 736e 0000 0000 0000 8f38
00000090 0a40 0000 0000
00000095
```

Рис. 21: Сигнатура заголовка базы данных в 16-ричном виде (hexdump)

Заключение

В ходе выполнения летней научно-исследовательской практики были полностью реализованы все поставленные задачи:

1. **Установка и настройка виртуальной среды Debian 12.14, проброс портов, беспарольная SSH-аутентификация:** Установлен Oracle VirtualBox, развернута виртуальная машина с Debian 12.14, настроен проброс 22-го порта для SSH-подключения, сгенерирована пара SSH-ключей и настроена беспарольная аутентификация.
2. **Освоение основ работы в командном интерпретаторе:** Изучены базовые команды Linux (*whoami*, *date*, *cal*, *uptime*, *df*, *cp*, *mv*, *touch*, *rm*, *stat*, *file*, *find*, *whereis*, *cat*, *hexdump*, *less*, *grep*, *time*), освоена работа с историей команд, автодополнением и редактированием командной строки.
3. **Разработка функции проверки палиндрома, сборка статической и динамической библиотек, реализация линковки через dlopen:** Реализована функция проверки палиндрома в двух вариантах (итеративном и рекурсивном) без использования библиотечных функций; созданы статическая (*libpalindrome.a*) и динамическая (*libpalindrome.so*) библиотеки с использованием CMake; реализована динамическая загрузка библиотеки через *dlopen()*. Средства разработки: *GCC 12.2.0*, *CMake*.
4. **Тестирование скорости работы при флагах оптимизации (O0, O1, O2, Os), сравнение размеров файлов:** Проведено тестирование производительности. Установлено, что флаг *-O2* обеспечивает максимальную производительность (ускорение до 16% по сравнению с *-O0*), а *-Os* дает наименьший размер исполняемого кода (1.8 КБ для статической библиотеки). Разница между статической и динамической линковкой составила

менее 1%.

5. **Исследование зависимостей с ldd:** С помощью команды *ldd* и разработанного скрипта *tree.sh* построены рекурсивные деревья зависимостей для программ из задания. Выявлено, что все программы зависят от стандартной библиотеки *libc.so.6* и динамического загрузчика *ld-linux-aarch64.so.1*. Программа с *dlopen* не показывает зависимость от *libpalindrome.so* в выводе *ldd*, так как библиотека загружается во время выполнения.
6. **Разработка структур PRICE и заголовка базы данных:** Создана структура *PRICE* с названием категории (8 байт, строка Си – указатель на строку с динамической памятью), названием товара (8 байт, строка Си – указатель на строку с динамической памятью), цена (8 байт, беззнаковое вещественное число с точностью до двух знаков после запятой), количество (4 байта, беззнаковое целочное число) и заголовок базы данных *DB_HEADER* с сигнатурой (4 байта, массив символов *char*), номером транзакции (4 байта, беззнаковое целое число), количеством записей (4 байта, беззнаковое целое число) и контрольной суммой CRC-32 (4 байта, беззнаковое целое число). Реализованы функции вычисления CRC-32, сохранения и загрузки базы данных в бинарный файл. *Библиотека: zlib*.
7. **Разработка приложения с псевдографическим интерфейсом ncurses:** Разработана программа на *ncurses* с главным меню, позволяющая:
 - добавлять товары с посимвольным вводом и валидацией данных;
 - выполнять поиск по категории с вычислением средней цены товара с максимальным количеством;
 - просматривать все записи базы данных;

- сохранять базу данных в бинарный файл;
- просматривать информацию о базе (сигнатура, транзакции, CRC-32).

Реализована защита от ввода некорректных данных (пустые поля, отрицательные значения, буквы в числовых полях). *Библиотека: ncurses.*

Все задачи, сформулированные в постановке, выполнены в полном объеме. Программный код соответствует требованиям индивидуального варианта №14.

Список используемых источников

- [1] Oracle Corporation. Oracle VirtualBox [Электронный ресурс] : официальный сайт. — URL: <https://www.oracle.com/virtualization/virtualbox/> (дата обращения: 18.06.2026).
- [2] КНИТУ-КАИ. МДК.01.04 Системное программирование. Лекция 21. Статические и динамические библиотеки [Электронный ресурс] : учебные материалы. — URL: <https://moodle.kstu.ru/course/section.php?id=69491> (дата обращения: 20.06.2026).
- [3] КНИТУ-КАИ. МДК.01.04 Системное программирование. Лекция 22. Подключение библиотеки во время выполнения: dlopen, dlsym, dlclose. Собственная библиотека [Электронный ресурс] : учебные материалы. — URL: <https://moodle.kstu.ru/course/section.php?id=69492> (дата обращения: 20.06.2026).
- [4] metanit.com. Язык программирования Си. Работа с файлами [Электронный ресурс] : учебный материал. — URL: <https://metanit.com/c/tutorial/10.1.php> (дата обращения: 21.06.2026).
- [5] Gookin D. Programming the Terminal with ncurses : [Электронный ресурс] / D. Gookin. — San Francisco : No Starch Press, 2007. — 576 p. — ISBN 978-1-59327-090-0.

Приложение А. Исходный код файлов задания №1

Листинг 1: Файл src/palindrome.h

```
1 #ifndef PALINDROME_H
2 #define PALINDROME_H
3
4 #include <stdbool.h>
5
6 bool palindrome(char *string, int n);
7 bool palindrome_recursive(char *string, int n);
8 int string_len(char *string);
9
10 #endif
```

Листинг 2: Файл src/palindrome.c

```
1 #include <stdbool.h> // false, true
2
3 bool palindrome(char *string, int n)
4 {
5     for (int i = 0; i < (int)(n / 2); i++)
6     {
7         if (string[i] != string[n - 1 - i])
8             return false;
9     }
10    return true;
11 }
12
13 bool palindrome_recursive(char *string, int n)
14 {
15     if (n == 0 || n == 1)
16         return true;
17     if (string[0] == string[n - 1])
18         return palindrome_recursive(string + 1, n - 2);
19     return false;
20 }
```

```

21
22 int string_len(char *string)
23 {
24     int n = 0;
25     char c = string[0];
26     while (c != '\0')
27         c = string[++n];
28     return n;
29 }

```

Листинг 3: Файл src/CmakeLists.txt

```

1  set(LIB_SOURCES palindrome.c)
2  set(LIB_HEADERS palindrome.h)
3
4  # статическая библиотека
5  if(BUILD_STATIC_LIB)
6      add_library(palindrome_static STATIC ${LIB_SOURCES})
7      target_include_directories(palindrome_static PUBLIC ${
CMAKE_CURRENT_SOURCE_DIR})
8      set_target_properties(palindrome_static PROPERTIES
9          OUTPUT_NAME palindrome
10         CLEAN_DIRECT_OUTPUT 1
11     )
12     install(TARGETS palindrome_static
13         ARCHIVE DESTINATION lib
14     )
15 endif()
16
17 # динамическая библиотека
18 if(BUILD_SHARED_LIB)
19     add_library(palindrome_shared SHARED ${LIB_SOURCES})
20     target_include_directories(palindrome_shared PUBLIC ${
CMAKE_CURRENT_SOURCE_DIR})
21     set_target_properties(palindrome_shared PROPERTIES
22         OUTPUT_NAME palindrome
23         CLEAN_DIRECT_OUTPUT 1
24         VERSION ${PROJECT_VERSION}

```

```

25     SOVERSION 1)
26
27     target_link_options(palindrome_shared PRIVATE
28         -Wl,-soname,libpalindrome.so.1
29
30     install(TARGETS palindrome_shared
31         LIBRARY DESTINATION lib
32         RUNTIME DESTINATION bin)
33 endif()
34
35 install(FILES ${LIB_HEADERS} DESTINATION include)

```

Листинг 4: Файл tests/test_palindrome.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <time.h>
5 #include "palindrome.h"
6
7 #define TEST_WORDS 1000000
8
9 void generate_random_string(char *str, int len)
10 {
11     for (int i = 0; i < len - 1; i++)
12         str[i] = 'a' + rand() % 26;
13     str[len - 1] = '\0';
14 }
15
16 double test_speed(int iterations, int word_len, bool use_recursive)
17 {
18     char *word = malloc(word_len);
19     clock_t start, end;
20     double cpu_time_used;
21     bool result;
22
23     start = clock();
24

```

```

25     for (int i = 0; i < iterations; i++)
26     {
27         generate_random_string(word, word_len);
28         int len = string_len(word);
29
30         if (use_recursive)
31             result = palindrome_recursive(word, len);
32         else
33             result = palindrome(word, len);
34         (void)result;
35     }
36
37     end = clock();
38     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
39
40     free(word);
41     return cpu_time_used;
42 }
43
44 int main(int argc, char *argv[])
45 {
46     int iterations = TEST_WORDS;
47     int word_len = 10000;
48     bool use_recursive = false;
49
50     if (argc > 1)
51         iterations = atoi(argv[1]);
52     if (argc > 2)
53         word_len = atoi(argv[2]) + 1; // +1 для '\0'
54     if (argc > 3 && strcmp(argv[3], "recursive") == 0)
55         use_recursive = true;
56
57     printf("Тестирование скорости:\n");
58     printf("Количество тестов: %d\n", iterations);
59     printf("Длина слова: %d\n", word_len - 1);
60     printf("Метод: %s\n", use_recursive ? "рекурсивный" : "итеративный");
61     printf("Тип линковки: %s\n",
62 #ifdef STATIC_BUILD

```



```

3 #include <string.h>
4 #include <time.h>
5
6 #include <dlfcn.h>
7 #include <stdbool.h>
8 #include <unistd.h>
9 #include <libgen.h>
10
11 #define TEST_WORDS 1000000
12
13 // определение типв указателя на функции
14 typedef bool (*palindrome_func_t)(char *, int);
15 typedef int (*string_len_func_t)(char *);
16
17 char *get_library_path()
18 {
19     static char lib_path[512];
20     char *lib_dir = getenv("PALINDROME_LIB_DIR");
21     if (lib_dir != NULL)
22     {
23         // формировка путь .so
24         snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.so", lib_dir);
25         return lib_path;
26     }
27
28     if (access("./libpalindrome.so", F_OK) == 0)
29     {
30         snprintf(lib_path, sizeof(lib_path), "./libpalindrome.so");
31         return lib_path;
32     }
33
34     char exe_path[512]; // буфер для пути к исполняемому файлу
35     ssize_t len = readlink("/proc/self/exe", exe_path, sizeof(exe_path) - 1);
36     if (len != -1)
37     {
38         exe_path[len] = '\0';
39         char *dir = dirname(exe_path);
40

```

```

41     snprintf(lib_path, sizeof(lib_path), "%s/../../lib/libpalindrome.so",
dir);
42     if (access(lib_path, F_OK) == 0)
43     {
44         return lib_path;
45     }
46     snprintf(lib_path, sizeof(lib_path), "%s/libpalindrome.so", dir);
47     return lib_path;
48 }
49
50
51     snprintf(lib_path, sizeof(lib_path), "libpalindrome.so");
52     return lib_path;
53 }
54
55 void generate_random_string(char *str, int len)
56 {
57     for (int i = 0; i < len - 1; i++)
58     {
59         str[i] = 'a' + rand() % 26;
60     }
61     str[len - 1] = '\0';
62 }
63
64 double test_speed(int iterations, int word_len, palindrome_func_t
palindrome_func, string_len_func_t len_func)
65 {
66     char *word = malloc(word_len);
67     clock_t start, end;
68     double cpu_time_used;
69     bool result;
70
71     start = clock();
72
73     for (int i = 0; i < iterations; i++)
74     {
75         generate_random_string(word, word_len);
76         int len = len_func(word);

```

```

77     result = palindrome_func(word, len);
78     (void)result;
79 }
80
81 end = clock();
82 cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
83
84 free(word);
85 return cpu_time_used;
86 }
87
88 int main(int argc, char *argv[])
89 {
90     void *handle; // указатель на загруженную библиотеку
91     palindrome_func_t palindrome_iter, palindrome_rec;
92     string_len_func_t string_len_func;
93     char *error;
94
95     char *lib_path = get_library_path();
96     handle = dlopen(lib_path, RTLD_LAZY); // загрузка библиотеки
97     if (!handle)
98     {
99         fprintf(stderr, "Ошибка загрузки библиотеки: %s\n", dlerror());
100         return 1;
101     }
102
103     printf("Библиотека успешно загружена\n");
104
105     // указатели на функции
106     palindrome_iter = (palindrome_func_t)dlsym(handle, "palindrome");
107     if ((error = dlerror()) != NULL)
108     {
109         fprintf(stderr, "Ошибка получения функции palindrome: %s\n", error);
110         dlclose(handle);
111         return 1;
112     }
113
114     palindrome_rec = (palindrome_func_t)dlsym(handle, "palindrome_recursive")

```

```

;
115 if ((error = dlerror()) != NULL)
116 {
117     fprintf(stderr, "Ошибка получения функции palindrome_recursive: %s\n",
error);
118     dlclose(handle);
119     return 1;
120 }
121
122 string_len_func = (string_len_func_t)dlsym(handle, "string_len");
123 if ((error = dlerror()) != NULL)
124 {
125     fprintf(stderr, "Ошибка получения функции string_len: %s\n", error);
126     dlclose(handle);
127     return 1;
128 }
129
130 int iterations = TEST_WORDS;
131 int word_len = 10000;
132
133 if (argc > 1)
134 {
135     iterations = atoi(argv[1]);
136 }
137 if (argc > 2)
138 {
139     word_len = atoi(argv[2]) + 1;
140 }
141
142 printf("\Тестирование скорости (dlopen):\n");
143 printf("Количество тестов: %d\n", iterations);
144 printf("Длина слова: %d\n", word_len - 1);
145
146 // тест итеративной функции
147 printf("\Итеративная функция:\n");
148 double time_iter = test_speed(iterations, word_len, palindrome_iter,
string_len_func);
149 printf("Время выполнения: %.6f секунд\n", time_iter);

```



```

3
4 # определение макроса STATIC_BUILD для статической версии
5 target_compile_definitions(test_palindrome PRIVATE STATIC_BUILD)
6
7 target_link_libraries(test_palindrome palindrome_static)
8 target_include_directories(test_palindrome PRIVATE ${CMAKE_SOURCE_DIR}/src)
9
10 add_test(NAME test_palindrome_static COMMAND test_palindrome 1000 10 iter) #
    тест в Ctest
11
12 # тест с динамической линковкой
13 add_executable(test_palindrome_dynamic test_palindrome.c)
14
15 # НЕ определяем STATIC_BUILD для динамической версии
16 target_link_libraries(test_palindrome_dynamic palindrome_shared)
17 target_include_directories(test_palindrome_dynamic PRIVATE ${CMAKE_SOURCE_DIR}
    }/src)
18
19 set_target_properties(test_palindrome_dynamic PROPERTIES
20     BUILD_RPATH "${CMAKE_LIBRARY_OUTPUT_DIRECTORY}"
21     INSTALL_RPATH "${CMAKE_INSTALL_PREFIX}/lib")

```

Листинг 7: Файл scripts/test_optimization.sh

```

1 echo "Тестирование оптимизаций компилятора с CMake :"
2 echo ""
3 # флаги оптимизации
4 OPTIMIZATIONS=("O0" "O1" "O2" "Os")
5 TEST_ITERATIONS=100000000
6 WORD_LENGTH=100000
7
8 NPROC=$(nproc 2>/dev/null || echo 4)
9
10 echo "Используется $NPROC ядер для сборки"
11 echo ""
12
13 # директория для результатов
14 mkdir -p results

```

```

15
16 for opt in "${OPTIMIZATIONS[@]}"; do
17     echo "Флаг оптимизации: -$opt"
18     echo "-----"
19
20     # директория для сборки
21     BUILD_DIR="build_$opt"
22     mkdir -p $BUILD_DIR
23     cd $BUILD_DIR
24
25     # конфигурация с флагом оптимизации
26     cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_C_FLAGS_RELEASE="- $opt "
27
28     # сборка
29     make clean > /dev/null 2>&1 # очищаем предыдущую сборку
30     make -j$NPROC > /dev/null 2>&1 # собираем проект
31
32     echo "Статическая линковка итеративный():"
33     time ./bin/test_palindrome $TEST_ITERATIONS $WORD_LENGTH iter
34
35     echo ""
36     echo "Статическая линковка рекурсивный():"
37     time ./bin/test_palindrome $TEST_ITERATIONS $WORD_LENGTH recursive
38
39     echo ""
40     echo "Динамическая линковка итеративный():"
41     time ./bin/test_palindrome_dynamic $TEST_ITERATIONS $WORD_LENGTH iter
42
43     echo ""
44     echo "Динамическая линковка рекурсивный():"
45     time ./bin/test_palindrome_dynamic $TEST_ITERATIONS $WORD_LENGTH
recursive
46
47     echo ""
48     echo "dlopen итеративный():"
49     export PALINDROME_LIB_DIR=$(pwd)/lib
50     time ./bin/test_dlopen $TEST_ITERATIONS $WORD_LENGTH
51

```

```

52     echo ""
53     # сохраняем размеры файлов
54     echo "Размеры файлов:" >> ../results/sizes_${opt}.txt
55     ls -lh bin/test_palindrome bin/test_palindrome_dynamic bin/test_dlopen
lib/*.a 2>/dev/null >> ../results/sizes_${opt}.txt
56     if [ -f "lib/libpalindrome.dylib" ]; then
57         ls -lh lib/libpalindrome.dylib >> ../results/sizes_${opt}.txt
58     fi
59
60     cd ..
61     echo ""
62 done

```

Листинг 8: Файл scripts/analyze_libs.sh

```

1 echo "Анализ разделяемых библиотек (CMake) :"
2 echo ""
3
4 # сборка в режиме release
5 mkdir -p build_analysis
6 cd build_analysis
7 cmake .. -DCMAKE_BUILD_TYPE=Release
8 make -j$(nproc) > /dev/null 2>&1
9
10 echo "1. Статическая линковка:"
11 ldd bin/test_palindrome
12 echo ""
13
14 echo "2. Динамическая линковка:"
15 ldd bin/test_palindrome_dynamic
16 echo ""
17
18 echo "3. Программа с dlopen:"
19 ldd bin/test_dlopen
20 echo ""
21
22 echo "4. Зависимости динамической библиотеки:"
23 ldd lib/libpalindrome.so

```



```

24 echo ""
25
26 echo "5. Размеры файлов:"
27 ls -lh bin/test_palindrome bin/test_palindrome_dynamic bin/test_dlopen lib/*.
    a lib/*.so
28 echo ""
29
30 cd ..
31 cd ..

```

Листинг 9: Файл CmakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(PalindromeProject VERSION 1.0 LANGUAGES C)
3
4 set(CMAKE_C_STANDARD 11)
5 set(CMAKE_C_STANDARD_REQUIRED ON)
6
7 option(BUILD_STATIC_LIB "Build static library" ON)
8 option(BUILD_SHARED_LIB "Build shared library" ON)
9 option(BUILD_TESTS "Build test executables" ON)
10 option(BUILD_DLOPEN_TEST "Build dlopen test" ON)
11
12 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
13 set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
14 set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
15
16 add_subdirectory(src)
17 if(BUILD_TESTS)
18     add_subdirectory(tests)
19 endif()
20
21 enable_testing()
22
23 message(STATUS "Project: ${PROJECT_NAME}")
24 message(STATUS "Version: ${PROJECT_VERSION}")
25 message(STATUS "Build type: ${CMAKE_BUILD_TYPE}")
26 message(STATUS "C Compiler: ${CMAKE_C_COMPILER}")

```

```
27 message(STATUS "Platform: ${CMAKE_SYSTEM_NAME}")
```

Приложение В. Исходный код файлов задания №2

Листинг 10: Файл src/price.h

```
1 #ifndef PRICE_H
2 #define PRICE_H
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <stdbool.h>
8 #include <zlib.h>      // CRC-32
9
10
11 typedef struct
12 {
13     char *category;
14     char *good;
15     double price;
16     unsigned int num;
17 } PRICE;
18
19 typedef struct
20 {
21     char signature[4];
22     unsigned int tr_num;
23     unsigned int st_count;
24     unsigned int crc32;
25 } DB_HEADER;
26
27 DB_HEADER create_header(const char *signature, unsigned int tx_num, unsigned
    int count, const PRICE *data);
28 unsigned int get_crc32(const PRICE *data, unsigned int count);
29 void save_database(const char *filename, DB_HEADER *header, PRICE *data,
    unsigned int count);
30 DB_HEADER load_database(const char *filename, PRICE **data);
```

```

31 void free_price(PRICE *item);
32
33
34 #endif

```

Листинг 11: Файл src/price.c

```

1  #include "price.h"
2  #include <stdio.h>
3  #include <zlib.h>
4  #include <stdbool.h> // false, true
5
6  unsigned int get_crc32(const PRICE *data, unsigned int count)
7  {
8      uLong crc = crc32(0L, Z_NULL, 0);
9
10     for (unsigned int i = 0; i < count; i++)
11     {
12         // category hash
13         crc = crc32(crc, (const Bytef *)data[i].category, strlen(data[i].
category));
14         // product name hash
15         crc = crc32(crc, (const Bytef *)data[i].good, strlen(data[i].good));
16         // price hash
17         crc = crc32(crc, (const Bytef *)&data[i].price, sizeof(double));
18         // quantity hash
19         crc = crc32(crc, (const Bytef *)&data[i].num, sizeof(unsigned int));
20     }
21
22     return (unsigned int)crc;
23 }
24
25 DB_HEADER create_header(const char *signature, unsigned int tr_num, unsigned
int count, const PRICE *data)
26 {
27     DB_HEADER header;
28     strncpy(header.signature, signature, 4);
29     // header.signature[4] = '\0';

```

```

30     header.tr_num = tr_num;
31     header.st_count = count;
32     header.crc32 = get_crc32(data, count);
33
34     return header;
35 }
36
37 void save_database(const char *filename, DB_HEADER *header, PRICE *data,
    unsigned int count)
38 {
39     FILE *file = fopen(filename, "wb");
40     if (!file)
41     {
42         fprintf(stderr, "Error opening file\n");
43         return;
44     }
45     fwrite(header, sizeof(DB_HEADER), 1, file);
46
47     for (unsigned int i = 0; i < count; i++)
48     {
49         unsigned int len = strlen(data[i].category) + 1;
50         fwrite(&len, sizeof(unsigned int), 1, file);
51         fwrite(data[i].category, sizeof(char), len, file);
52
53         len = strlen(data[i].good) + 1;
54         fwrite(&len, sizeof(unsigned int), 1, file);
55         fwrite(data[i].good, sizeof(char), len, file);
56
57         fwrite(&data[i].price, sizeof(double), 1, file);
58         fwrite(&data[i].num, sizeof(unsigned int), 1, file);
59     }
60     fclose(file);
61     printf("Database saved to file: %s\n", filename);
62     printf("Records count: %u\n", count);
63     printf("CRC-32: 0x%08X\n", header->crc32);
64 }
65
66 DB_HEADER load_database(const char *filename, PRICE **data)

```

```

67 {
68     DB_HEADER header = {0};
69     FILE *file = fopen(filename, "rb");
70
71     if (!file)
72     {
73         fprintf(stderr, "Error opening file\n");
74         return header;
75     }
76
77     size_t read = fread(&header, sizeof(DB_HEADER), 1, file);
78     if (read != 1)
79     {
80         fprintf(stderr, "Error reading header\n");
81         fclose(file);
82         return header;
83     }
84
85     *data = (PRICE *)malloc(header.st_count * sizeof(PRICE));
86     if (!*data)
87     {
88         fprintf(stderr, "Memory allocation error\n");
89         fclose(file);
90         return header;
91     }
92
93     for (unsigned int i = 0; i < header.st_count; i++)
94     {
95         unsigned int len;
96         char *str;
97
98         fread(&len, sizeof(unsigned int), 1, file);
99         str = (char *)malloc(len * sizeof(char));
100        fread(str, sizeof(char), len, file);
101        (*data)[i].category = str;
102
103        fread(&len, sizeof(unsigned int), 1, file);
104        str = (char *)malloc(len * sizeof(char));

```

```

105     fread(str, sizeof(char), len, file);
106     (*data)[i].good = str;
107
108     fread(&(*data)[i].price, sizeof(double), 1, file);
109     fread(&(*data)[i].num, sizeof(unsigned int), 1, file);
110 }
111
112 fclose(file);
113 printf("Database loaded from file: %s\n", filename);
114 printf("Records count: %u\n", header.st_count);
115 printf("Transaction number: %u\n", header.tr_num);
116 printf("CRC-32: 0x%08X\n", header.crc32);
117
118 return header;
119 }
120
121 void free_price(PRICE *item)
122 {
123     if (item->category)
124     {
125         free(item->category);
126         item->category = NULL;
127     }
128     if (item->good)
129     {
130         free(item->good);
131         item->good = NULL;
132     }
133 }

```

Листинг 12: Файл src/main.c

```

1 #include <ncurses.h>
2 #include <ctype.h>
3 #include "price.h"
4
5 PRICE *items = NULL;
6 unsigned int item_count = 0;

```

```

7 DB_HEADER header;
8 char filename[256] = "databasetest.bin";
9
10 int do_main = 1;
11
12 int input_data_ncurses();
13 int search_category_ncurses();
14 void search_by_category(const char *category);
15 void show_all_products();
16 void show_database_info();
17 void save_with_message(char *signature);
18 void draw_main_menu(WINDOW *win, const char *status_msg);
19 int get_menu_choice(WINDOW *win);
20
21 void draw_main_menu(WINDOW *win, const char *status_msg)
22 {
23     werase(win);
24     box(win, 0, 0);
25     mvwprintw(win, 0, 2, " PRODUCT DATABASE MANAGEMENT ");
26
27     mvwprintw(win, 2, 4, "1. Add new product");
28     mvwprintw(win, 3, 4, "2. Search by category");
29     mvwprintw(win, 4, 4, "3. Show all products");
30     mvwprintw(win, 5, 4, "4. Save database");
31     mvwprintw(win, 6, 4, "5. Database info");
32     mvwprintw(win, 7, 4, "6. Exit");
33
34     mvwprintw(win, 9, 4, "Records in memory: %u", item_count);
35     mvwprintw(win, 10, 4, "Transaction #: %u", header.tr_num);
36
37     if (status_msg && strlen(status_msg) > 0)
38     {
39         mvwprintw(win, 12, 4, "%s CRC-32: 0x%08X", status_msg, header.crc32);
40     }
41     mvwprintw(win, 14, 4, "Select action (1-6): ");
42     wrefresh(win);
43 }
44

```



```

45 int get_menu_choice(WINDOW *win)
46 {
47     char input[256];
48     int choice;
49
50     while (1)
51     {
52         mvwprintw(win, 14, 24, "                ");
53         wmove(win, 14, 24);
54         wrefresh(win);
55
56         echo();
57         wgetnstr(win, input, sizeof(input) - 1);
58         noecho();
59
60         input[strcspn(input, "\n")] = '\0';
61
62         int valid_input = 1;
63         if (input[0] == '\0')
64             valid_input = 0;
65         else
66         {
67             for (int i = 0; input[i] != '\0'; i++)
68             {
69                 if (!isdigit(input[i]))
70                 {
71                     valid_input = 0;
72                     break;
73                 }
74             }
75         }
76
77         if (valid_input)
78         {
79             choice = atoi(input);
80             if (choice >= 1 && choice <= 6)
81                 return choice;
82         }

```

```

83
84     mvwprintw(win, 15, 4, "Invalid input! Please enter number 1-6.");
85     wrefresh(win);
86     napms(1000);
87     mvwprintw(win, 15, 4, "                                     ");
88     wrefresh(win);
89 }
90 }
91
92 int input_data_ncurses()
93 {
94     echo();
95     curs_set(1);
96
97     int height = 14;
98     int width = 55;
99     int start_y = 2;
100    int start_x = 10;
101
102    WINDOW *win = newwin(height, width, start_y, start_x);
103    box(win, 0, 0);
104    mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
105    mvwprintw(win, 13, 2, "Press ESC to cancel");
106    wrefresh(win);
107
108    char category[256] = "";
109    char good[256] = "";
110    char price_str[256] = "";
111    char num_str[256] = "";
112
113    int valid = 0;
114    int cancelled = 0;
115    double price = 0;
116    unsigned int num = 0;
117
118    while (!valid && !cancelled)
119    {
120        werase(win);

```

```

121     box(win, 0, 0);
122     mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
123     mvwprintw(win, 13, 2, "Press ESC to cancel");
124     wrefresh(win);
125
126     mvwprintw(win, 2, 2, "Category: ");
127     wmove(win, 2, 14);
128     wclrtoeol(win);
129     wrefresh(win);
130
131     int ch;
132     int pos = 0;
133     while (1)
134     {
135         ch = wgetch(win);
136         if (ch == 27) // esc
137         {
138             cancelled = 1;
139             break;
140         }
141         if (ch == 10) // enter
142         {
143             category[pos] = '\0';
144             break;
145         }
146         if (ch == KEY_BACKSPACE || ch == 127) // backspace or delete
147         {
148             if (pos > 0)
149             {
150                 pos--;
151                 category[pos] = '\0';
152                 mvwprintw(win, 2, 14, "%-30s", category);
153                 wmove(win, 2, 14 + pos);
154                 wrefresh(win);
155             }
156             continue;
157         }
158         if (pos < 255 && ch >= 32 && ch <= 126) // разрешены только цифры и

```

точка

```
159         {
160             category[pos++] = ch;
161             category[pos] = '\0';
162             mvwprintw(win, 2, 14, "%s", category);
163             wrefresh(win);
164         }
165     }
166     if (cancelled)
167         break;
168
169     mvwprintw(win, 4, 2, "Product name: ");
170     wmove(win, 4, 16);
171     wclrtoeol(win);
172     wrefresh(win);
173
174     pos = 0;
175     while (1)
176     {
177         ch = wgetch(win);
178         if (ch == 27)
179         {
180             cancelled = 1;
181             break;
182         }
183         if (ch == 10)
184         {
185             good[pos] = '\0';
186             break;
187         }
188         if (ch == KEY_BACKSPACE || ch == 127)
189         {
190             if (pos > 0)
191             {
192                 pos--;
193                 good[pos] = '\0';
194                 mvwprintw(win, 4, 16, "%-30s", good);
195                 wmove(win, 4, 16 + pos);
```

```

196         wrefresh(win);
197     }
198     continue;
199 }
200 if (pos < 255 && ch >= 32 && ch <= 126)
201 {
202     good[pos++] = ch;
203     good[pos] = '\0';
204     mvwprintw(win, 4, 16, "%s", good);
205     wrefresh(win);
206 }
207 }
208 if (cancelled)
209     break;
210
211 mvwprintw(win, 6, 2, "Cost (rub): ");
212 wmove(win, 6, 14);
213 wclrtoeol(win);
214 wrefresh(win);
215
216 pos = 0;
217 while (1)
218 {
219     ch = wgetch(win);
220     if (ch == 27)
221     {
222         cancelled = 1;
223         break;
224     }
225     if (ch == 10)
226     {
227         price_str[pos] = '\0';
228         break;
229     }
230     if (ch == KEY_BACKSPACE || ch == 127)
231     {
232         if (pos > 0)
233             {

```

```

234         pos--;
235         price_str[pos] = '\0';
236         mvwprintw(win, 6, 14, "%-30s", price_str);
237         wmove(win, 6, 14 + pos);
238         wrefresh(win);
239     }
240     continue;
241 }
242 if (pos < 255 && ((ch >= '0' && ch <= '9') || ch == '.'))
243 {
244     price_str[pos++] = ch;
245     price_str[pos] = '\0';
246     mvwprintw(win, 6, 14, "%s", price_str);
247     wrefresh(win);
248 }
249 }
250 if (cancelled)
251     break;
252
253 mvwprintw(win, 8, 2, "Amount: ");
254 wmove(win, 8, 10);
255 wclrtoeol(win);
256 wrefresh(win);
257
258 pos = 0;
259 while (1)
260 {
261     ch = wgetch(win);
262     if (ch == 27)
263     {
264         cancelled = 1;
265         break;
266     }
267     if (ch == 10)
268     {
269         num_str[pos] = '\0';
270         break;
271     }

```

```

272         if (ch == KEY_BACKSPACE || ch == 127)
273         {
274             if (pos > 0)
275             {
276                 pos--;
277                 num_str[pos] = '\0';
278                 mvwprintw(win, 8, 10, "%-30s", num_str);
279                 wmove(win, 8, 10 + pos);
280                 wrefresh(win);
281             }
282             continue;
283         }
284         if (pos < 255 && ch >= '0' && ch <= '9')
285         {
286             num_str[pos++] = ch;
287             num_str[pos] = '\0';
288             mvwprintw(win, 8, 10, "%s", num_str);
289             wrefresh(win);
290         }
291     }
292     if (cancelled)
293         break;
294
295     // валидация
296     int valid_price = 1;
297     int valid_num = 1;
298     int has_dot = 0;
299
300     if (strlen(category) == 0 || strlen(good) == 0)
301     {
302         valid = 0;
303         mvwprintw(win, 11, 2, "ERROR: Category and Product name cannot be
empty!");
304         wrefresh(win);
305         getch(); // ожидание нажатия клавиши
306         continue;
307     }
308

```

```

309     if (strlen(price_str) == 0)
310     {
311         valid_price = 0;
312     }
313     else
314     {
315         for (int i = 0; price_str[i] != '\0'; i++)
316         {
317             if (price_str[i] == '.')
318             {
319                 if (has_dot)
320                 {
321                     valid_price = 0;
322                     break;
323                 }
324                 has_dot = 1;
325             }
326             else if (!isdigit(price_str[i]))
327             {
328                 valid_price = 0;
329                 break;
330             }
331         }
332     }
333
334     if (strlen(num_str) == 0)
335         valid_num = 0;
336     else
337     {
338         for (int i = 0; num_str[i] != '\0'; i++)
339         {
340             if (!isdigit(num_str[i]))
341             {
342                 valid_num = 0;
343                 break;
344             }
345         }
346     }

```



```

347
348     if (valid_price && valid_num)
349     {
350         price = atof(price_str);
351         num = atoi(num_str);
352
353         if (price <= 0 || num == 0)
354         {
355             valid = 0;
356             if (price <= 0)
357                 mvwprintw(win, 11, 2, "ERROR: Price must be greater than
358 0!");
359             else
360                 mvwprintw(win, 11, 2, "ERROR: Amount must be greater than
361 0!");
362             wrefresh(win);
363             getch();
364             continue;
365         }
366         valid = 1;
367     }
368     else
369     {
370         valid = 0;
371         if (!valid_price)
372             mvwprintw(win, 11, 2, "ERROR: Price must be a positive number
373 (e.g. 99.50)");
374         else if (!valid_num)
375             mvwprintw(win, 11, 2, "ERROR: Amount must be a positive
376 integer (e.g. 10)");
377         wrefresh(win);
378         getch();
379     }
380
381     if (cancelled)
382     {

```

```

381     delwin(win);
382     noecho();
383     curs_set(0);
384     return 0;
385 }
386
387 items = (PRICE *)realloc(items, (item_count + 1) * sizeof(PRICE));
388 if (items)
389 {
390     items[item_count].category = strdup(category);
391     items[item_count].good = strdup(good);
392     items[item_count].price = price;
393     items[item_count].num = num;
394     item_count++;
395
396     werase(win);
397     box(win, 0, 0);
398     mvwprintw(win, 0, 2, " ENTER PRODUCT DATA ");
399     mvwprintw(win, 5, 2, "Product added successfully!");
400     mvwprintw(win, 7, 2, "Category: %s", category);
401     mvwprintw(win, 8, 2, "Name: %s", good);
402     mvwprintw(win, 9, 2, "Price: %.2f rub.", price);
403     mvwprintw(win, 10, 2, "Amount: %u", num);
404     mvwprintw(win, 12, 2, "Press any key to continue...");
405     wrefresh(win);
406     getch();
407
408     delwin(win);
409     noecho();
410     curs_set(0);
411     return 1;
412 }
413
414 delwin(win);
415 noecho();
416 curs_set(0);
417 return 0;
418 }

```

```

419
420 int search_category_ncurses()
421 {
422     char category[256] = "";
423     int rows, cols;
424     WINDOW *win;
425     int ch;
426     int pos = 0;
427     int cancelled = 0;
428
429     curs_set(1);
430     echo();
431
432     getmaxyx(stdscr, rows, cols);
433
434     win = newwin(7, 50, (rows - 7) / 2, (cols - 50) / 2);
435     box(win, 0, 0);
436     mvwprintw(win, 0, 2, " SEARCH BY CATEGORY ");
437     mvwprintw(win, 2, 2, "Enter category: ");
438     mvwprintw(win, 5, 2, "Press ESC to cancel");
439     wrefresh(win);
440
441     mvwprintw(win, 2, 18, "
442     wmove(win, 2, 18);
443     wrefresh(win);
444
445     while (1)
446     {
447         ch = wgetch(win);
448         if (ch == 27)
449         { // esc
450             cancelled = 1;
451             break;
452         }
453         if (ch == 10)
454         { // enter
455             category[pos] = '\0';
456             break;

```

```

457     }
458     if (ch == KEY_BACKSPACE || ch == 127)
459     {
460         if (pos > 0)
461         {
462             pos--;
463             category[pos] = '\0';
464             mvwprintw(win, 2, 18, "%-30s", category);
465             wmove(win, 2, 18 + pos);
466             wrefresh(win);
467         }
468         continue;
469     }
470     if (pos < 255 && ch >= 32 && ch <= 126)
471     {
472         category[pos++] = ch;
473         category[pos] = '\0';
474         mvwprintw(win, 2, 18, "%s", category);
475         wrefresh(win);
476     }
477 }
478
479 delwin(win);
480 noecho();
481 curs_set(0);
482
483 if (cancelled || strlen(category) == 0)
484 {
485     return 0; // отмена
486 }
487
488 search_by_category(category);
489 return 1; // успешно
490 }
491
492 void search_by_category(const char *category)
493 {
494     bool found = false;

```

```

495 unsigned int max_num = 0;
496 unsigned int count_found = 0;
497 double total_price = 0.0;
498 int indices[100] = {0};
499
500 for (unsigned int i = 0; i < item_count; i++)
501 {
502     if (strcmp(items[i].category, category) == 0)
503     {
504         found = true;
505         if (items[i].num > max_num)
506         {
507             max_num = items[i].num;
508             count_found = 1;
509             total_price = items[i].price;
510             indices[0] = i;
511         }
512         else if (items[i].num == max_num)
513         {
514             indices[count_found++] = i;
515             total_price += items[i].price;
516         }
517     }
518 }
519
520 clear();
521 refresh();
522 box(stdscr, 0, 0);
523
524 if (!found)
525 {
526     mvprintw(5, 10, "No products found in category: %s", category);
527 }
528 else
529 {
530     mvprintw(2, 10, "SEARCH RESULTS ");
531     mvprintw(3, 10, "Category: %s", category);
532     mvprintw(5, 10, "Maximum quantity: %u", max_num);

```

```

533     mvprintw(6, 10, "Number of products with max num: %u", count_found);
534     mvprintw(7, 10, "Average price: %.2f rub.", total_price / count_found
);
535
536     mvprintw(9, 10, "Products with max quantity:");
537     for (unsigned int i = 0; i < count_found; i++)
538     {
539         int idx = indices[i];
540         mvprintw(11 + i, 15, "%s - %.2f rub.", items[idx].good, items[idx
].price);
541     }
542 }
543
544 mvprintw(20, 10, "Press any key to continue...");
545 refresh();
546 getch();
547
548 }
549
550 void show_all_products()
551 {
552     clear();
553     refresh();
554
555     int rows, cols;
556     getmaxyx(stdscr, rows, cols);
557
558     if (item_count == 0)
559     {
560         mvprintw(rows / 2, (cols - 20) / 2, "Database is empty!");
561         getch();
562         endwin();
563         return;
564     }
565
566     box(stdscr, 0, 0);
567     mvprintw(0, 2, " ALL PRODUCTS ");
568

```

```

569     int line = 2;
570     for (unsigned int i = 0; i < item_count && line < rows - 2; i++)
571     {
572         mvprintw(line++, 2, "[%u] Category: %s", i + 1, items[i].category);
573         mvprintw(line++, 4, "Name: %s", items[i].good);
574         mvprintw(line++, 4, "Price: %.2f rub.", items[i].price);
575         mvprintw(line++, 4, "Amount: %u", items[i].num);
576         line++;
577     }
578
579     mvprintw(rows - 2, 2, "Total: %u products. Press any key...", item_count)
580 ;
581     refresh();
582     getch();
583 }
584
585 void show_database_info()
586 {
587     clear();
588     refresh();
589
590     int rows, cols;
591     getmaxyx(stdscr, rows, cols);
592
593     box(stdscr, 0, 0);
594     mvprintw(0, 2, " DATABASE INFO ");
595
596     mvprintw(2, 4, "Signature: %.4s", header.signature);
597     mvprintw(3, 4, "Transaction: %u", header.tr_num);
598     mvprintw(4, 4, "Records in file: %u", header.st_count);
599     mvprintw(5, 4, "Records in memory: %u", item_count);
600     mvprintw(6, 4, "CRC-32: 0x%08X", header.crc32);
601     mvprintw(7, 4, "File: %s", filename);
602
603     mvprintw(rows - 2, 4, "Press any key to continue...");
604     refresh();
605     getch();
606 }

```

```

606
607 void save_with_message(char *signature)
608 {
609     if (item_count == 0)
610     {
611         initscr();
612         clear();
613         refresh();
614         box(stdscr, 0, 0);
615         mvprintw(5, 10, "No data to save!");
616         mvprintw(7, 10, "Press any key...");
617         refresh();
618         getch();
619         endwin();
620         return;
621     }
622
623     header.st_count = item_count;
624     header = create_header(signature, header.tr_num, item_count, items);
625     save_database(filename, &header, items, item_count);
626
627     clear();
628     refresh();
629     box(stdscr, 0, 0);
630     mvprintw(3, 10, "Database saved successfully!");
631     mvprintw(4, 10, "Records: %u", item_count);
632     mvprintw(5, 10, "CRC-32: 0x%08X", header.crc32);
633     mvprintw(7, 10, "Press any key...");
634     refresh();
635     getch();
636 }
637
638 int main()
639 {
640     char *signature = "TASH";
641     int choice;
642     char status_msg[256] = "";
643     WINDOW *main_win;

```



```

644     int rows, cols;
645
646     initscr();
647     cbreak();
648     noecho();
649     keypad(stdscr, TRUE);
650     curs_set(0);
651
652     // загрузка базы данных
653     PRICE *loaded_data = NULL;
654     header = load_database(filename, &loaded_data);
655
656     if (loaded_data != NULL && header.st_count > 0)
657     {
658         items = loaded_data;
659         item_count = header.st_count;
660         snprintf(status_msg, sizeof(status_msg), "Loaded %u records.",
item_count);
661     }
662     else
663     {
664         memset(&header, 0, sizeof(DB_HEADER));
665         strncpy(header.signature, signature, 4);
666         header.tr_num = 0;
667         header.st_count = 0;
668         header.crc32 = 0;
669         snprintf(status_msg, sizeof(status_msg), "No database found. Creating
new.");
670     }
671
672     while (do_main)
673     {
674         initscr();
675         cbreak();
676         noecho();
677         keypad(stdscr, TRUE);
678         curs_set(0);
679         clear();

```

```

680     refresh();
681
682     getmaxyx(stdscr, rows, cols);
683
684     main_win = newwin(17, 55, (rows - 17) / 2, (cols - 55) / 2);
685     draw_main_menu(main_win, status_msg);
686
687     curs_set(1);
688     choice = get_menu_choice(main_win);
689
690     delwin(main_win);
691     endwin();
692
693     switch (choice)
694     {
695     case 1:
696     {
697         int added = input_data_ncurses();
698         if (added)
699         {
700             header.tr_num++;
701             snprintf(status_msg, sizeof(status_msg),
702                     "Product added. Transaction #%u", header.tr_num);
703             printf("Product added. Transaction #%u, CRC-32: 0x%08X",
704 header.tr_num, header.crc32);
705         }
706         else
707         {
708             snprintf(status_msg, sizeof(status_msg),
709                     "Product addition cancelled.");
710         }
711         break;
712     }
713
714     case 2:
715         if (item_count == 0)
716         {
717             snprintf(status_msg, sizeof(status_msg),

```

```

717         "Database is empty.");
718     break;
719 }
720
721     int search_result = search_category_ncurses();
722     if (search_result == 1)
723     {
724         header.tr_num++;
725         snprintf(status_msg, sizeof(status_msg),
726                 "Search completed. Transaction #%u", header.tr_num);
727         printf("Search completed. Transaction #%u, CRC-32: 0x%08X",
header.tr_num, header.crc32);
728     }
729     else
730     {
731         snprintf(status_msg, sizeof(status_msg),
732                 "Search cancelled.");
733     }
734     break;
735
736     case 3:
737         show_all_products();
738         snprintf(status_msg, sizeof(status_msg),
739                 "Displayed all %u products.", item_count);
740         break;
741         printf("Displayed all %u products, CRC-32: 0x%08X", item_count,
header.crc32);
742
743     case 4:
744         save_with_message(signature);
745         snprintf(status_msg, sizeof(status_msg),
746                 "Database saved successfully.");
747         printf("Database saved successfully, CRC-32: 0x%08X", header.
crc32);
748         break;
749
750     case 5:
751         show_database_info();

```

```

752     snprintf(status_msg, sizeof(status_msg),
753              "Database info displayed.");
754     printf("Database info displayed, CRC-32: 0x%08X", header.crc32);
755
756     break;
757
758     case 6:
759         if (item_count > 0)
760         {
761             header.st_count = item_count;
762             header = create_header(signature, header.tr_num, item_count,
items);
763             save_database(filename, &header, items, item_count);
764         }
765         for (unsigned int i = 0; i < item_count; i++)
766         {
767             free_price(&items[i]);
768         }
769         if (items)
770             free(items);
771
772         return 0;
773
774     default:
775         snprintf(status_msg, sizeof(status_msg),
776                  "Invalid choice. Please select 1-6.");
777         break;
778     }
779 }
780
781     endwin();
782     return 0;
783 }

```

Листинг 13: Файл CmakeLists.txt

```

1 cmake_minimum_required(VERSION 3.20)
2 project(PriceDB)

```

```
3
4 set(CMAKE_C_STANDARD 11)
5
6 set(SOURCES
7     src/main.c
8     src/price.c
9 )
10
11 add_executable(price_db ${SOURCES})
12
13 target_include_directories(price_db PRIVATE src)
14
15 find_package(ZLIB REQUIRED)
16
17 target_link_libraries(price_db
18     ncurses
19     ${ZLIB_LIBRARIES}
20     m
21 )
```